8

Fast multiplication

In this chapter, we introduce fast methods for multiplying integers and polynomials. We start with a simple method due to Karatsuba which reduces the cost from the classical $O(n^2)$ for polynomials of degree n to $O(n^{1.59})$. The Discrete Fourier Transform and its efficient implementation, the Fast Fourier Transform, are the backbone of the fastest algorithms. These work only when appropriate **roots of unity** are present, but Schönhage & Strassen (1971) showed how to create "virtual" roots that lead to a multiplication cost of only $O(n \log n \log\log n)$. In Chapter 9, Newton iteration will help us extend this to fast division with remainder.

General-purpose computer algebra systems typically only implement the classical method, and sometimes Karatsuba's. This is quite sufficient as long as one deals with fairly small numbers or polynomials, but for many high-performance tasks fast arithmetic is indispensable. Examples include factoring large polynomials (Section 15.7), finding primes and twin primes (Notes to Chapter 18), and computing billions of digits of π (Section 4.6) or billions of roots of Riemann's zeta function (Notes 18.4).

Asymptotically fast methods are standard tools in many areas of computer science, where, say, $O(n \log n)$ sorting algorithms like quicksort or mergesort are widely used and experiments show that they outperform the "classical" $O(n^2)$ sorting algorithms like bubble sort or insertion sort already for values of n below 100. In contrast, the asymptotically fast algorithms for polynomial and integer arithmetic, in particular for multiplication, have received until recently comparatively little attention in the computer algebra world since their invention around 1970. Some of the reasons may be that the fast algorithms are often considerably more complicated than the classical ones, and that the crossover points between these algorithms may be disappointingly high when the algorithms are implemented "literally" as described in textbooks, without any further optimization. On the other hand, experiments with highly optimized software, such as those described in Section 9.7, show that Karatsuba's algorithm has a fairly small crossover with the classical algorithm, and that even faster multiplication algorithms already come into play for moderately sized inputs. Designers of a computer algebra system

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

should carefully determine the crossover points and then, depending on the size of the problems their system is intended to solve, decide which algorithms to offer.

Last but not least there is the intellectual beauty of asymptotic analysis. Complexity theory provides a precise framework in which to compare algorithms via their asymptotic running time (or some other measure, such as memory or parallel time). For our problems, both *Boolean* and *arithmetic complexity theory* play a role; Bürgisser, Clausen & Shokrollahi (1997) give an overview of the impressive results of the latter. It provides tools for proving *lower bounds*, saying that every conceivable algorithm must use at least so and so many operations, and in lucky cases even that some algorithms are *optimal*. Furthermore, this crystal-clear framework allows for a precise statement that "this new method makes progress" and, if incorrect, its refutation. Practical results, such as the experiments reported in this book, are also important, but often open to dispute as they take place in the muddy waters of difficult-to-compare computing environments, difference of opinion on the "important cases and examples", and difficulty to reproduce. The latter is extremely laborious or virtually impossible for a large implementation effort. As an example, Fürer (2009) improved the $O(n \log n \operatorname{loglog} n)$ multiplication algorithm of Schönhage & Strassen (1971); it is difficult to imagine this status of a well-defined challenge for over a quarter century for an experimental problem. Some areas have an accepted set of concrete benchmark problems, such as the "most wanted" Cunningham numbers for integer factorization (see Chapter 19). But even there, asymptotic progress is the holy grail.

The lower table on the inside back cover lists the problems in polynomial algebra for which we will achieve almost linear-time algorithms in the following chapters. The algorithms work over an arbitrary ring or field, and fast polynomial multiplication is crucial for them. For all these problems, the input size is about n, and the classical algorithms from Part I take quadratic time. All algorithms have integer analogs, which—as always—are more complicated due to the carries, with about the same running time in word operations when the input consists of n words.

8.1. Karatsuba's multiplication algorithm

We start with the multiplication of two polynomials $f, g \in R[x]$ of degree less than n over a ring R. As usual, "ring" means a commutative ring with 1. If $f_i, g_j, h_k \in R$ are the coefficients of f, g and $h = fg$, respectively, the classical multiplication algorithm uses $O(n^2)$ operations in R to compute the h_k from the f_i and g_j: n^2 multiplications $f_i g_j$ plus $(n-1)^2$ additions for all $h_k = \sum_{i+j=k} f_i g_j$ (Section 2.3). For instance, multiplying $(ax+b)(cx+d) = acx^2 + (ad+bc)x + bd$ uses four multiplications ac, ad, bc, bd, and one addition $ad+bc$.

Surprisingly, there is an easy method of doing better. We compute ac, bd, $u = (a+b)(c+d)$, and $ad+bc = u - ac - bd$, with three multiplications and four additions and subtractions. The total has increased to seven operations, but a

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

recursive application will drastically reduce the overall cost (see Figure 8.2). To explain the general approach, we assume that $n = 2^k$ for some $k \in \mathbb{N}$, set $m = n/2$, and rewrite f and g in the form $f = F_1 x^m + F_0$ with $F_0, F_1 \in R[x]$ of degree less than m and similarly $g = G_1 x^m + G_0$. (If $\deg f < n-1$, then some of the top coefficients are zero.) Now $fg = F_1 G_1 x^n + (F_0 G_1 + F_1 G_0) x^m + F_0 G_0$. In this form, multiplication of f and g has been reduced to four multiplications of polynomials of degree less than m. Multiplication by a power of x does not count as a multiplication, since it corresponds merely to a shift of the coefficients.

So far we have not really achieved anything. But the method by Karatsuba in Karatsuba & Ofman (1962), explained for $n = 1$ above, shows how this expression for fg can be rearranged to reduce the number of multiplications of the smaller polynomials at the expense of increasing the number of additions. Since multiplication is slower than addition, a saving is obtained when n is sufficiently large. We rewrite the product as $fg = F_1 G_1 x^n + ((F_0 + F_1)(G_0 + G_1) - F_0 G_0 - F_1 G_1) x^m + F_0 G_0$. This expression shows that multiplication of f and g requires only three multiplications of polynomials of degree less than m and some additions. The same method is now applied recursively to the smaller multiplications. If $T(n)$ denotes the time necessary to multiply two polynomials of degree less than n, then $T(2n) \leq 3T(n) + cn$, for some constant c. The linear term comes from the observation that addition of two polynomials of degree less than d can be done with d operations in R.

Here is the corresponding algorithm.

ALGORITHM 8.1 Karatsuba's polynomial multiplication algorithm.
Input: $f, g \in R[x]$ of degrees less than n, where R is a ring (commutative, with 1) and n a power of 2.
Output: $fg \in R[x]$.

1. **if** $n = 1$ **then return** $f \cdot g \in R$
2. let $f = F_1 x^{n/2} + F_0$ and $g = G_1 x^{n/2} + G_0$, with $F_0, F_1, G_0, G_1 \in R[x]$ of degrees less than $n/2$
3. compute $F_0 G_0$, $F_1 G_1$, and $(F_0 + F_1)(G_0 + G_1)$ by a recursive call
4. **return** $F_1 G_1 x^n + ((F_0 + F_1)(G_0 + G_1) - F_0 G_0 - F_1 G_1) x^{n/2} + F_0 G_0$

Figure 8.1 visualizes this algorithm in the form of an arithmetic circuit for $n = 4$.

We first need a lemma which will be helpful in the analysis of several recursive algorithms. $T(n)$ will be the cost on input size n of the algorithm, which consists of b recursive calls with inputs of size $n/2$, plus some cost denoted by $S(n)$. We denote by log the binary logarithm.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

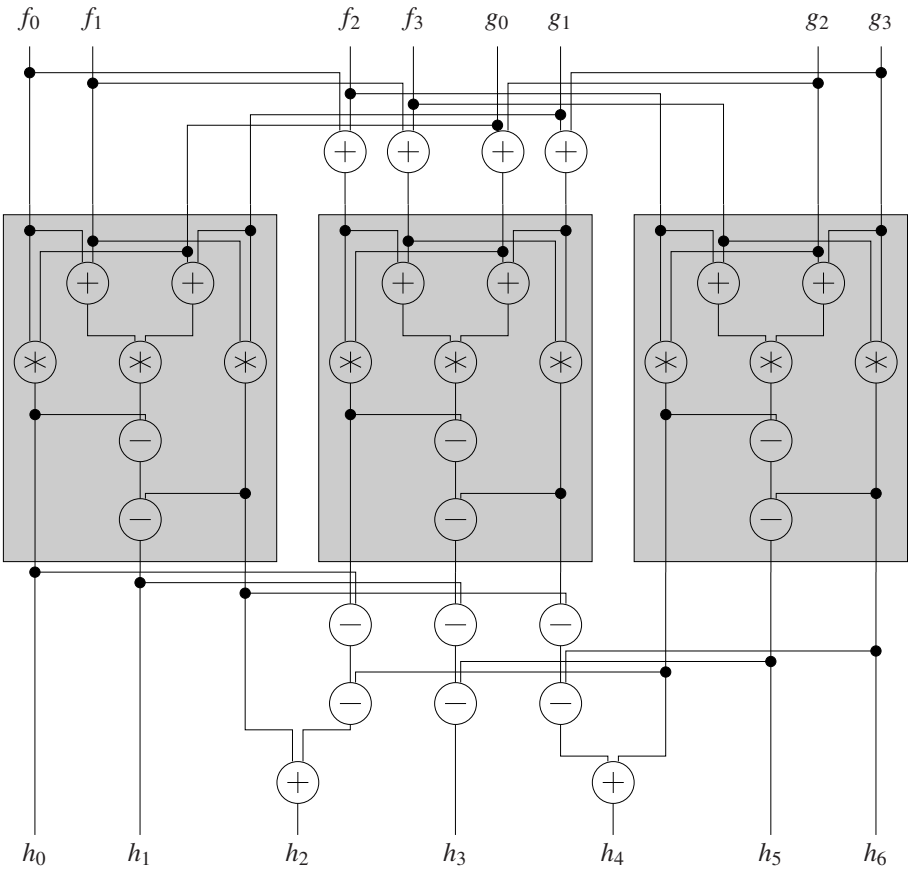


FIGURE 8.1: An arithmetic circuit illustrating Karatsuba's algorithm for $n = 4$. The shaded boxes are Karatsuba circuits for $n = 2$. A subtraction node computes the difference of its left input minus its right input. The flow of control is from top do bottom.

LEMMA 8.2. *Let $b, c \in \mathbb{R}_{>0}$, $d \in \mathbb{R}_{\geq 0}$, $S, T : \mathbb{N}_{\geq 1} \longrightarrow \mathbb{N}_{\geq 1}$ be functions such that $S(2n) \geq cS(n)$ for all $n \in \mathbb{N}_{\geq 1}$, and*

$$T(1) = d, \quad T(n) \leq bT(n/2) + S(n) \text{ for } n = 2^i \text{ and } i \in \mathbb{N}_{\geq 1}.$$

Then for $i \in \mathbb{N}$ and $n = 2^i$ we have

$$T(n) \leq \begin{cases} dn^{\log b} + S(n) \log n & \text{if } b = c, \\ dn^{\log b} + \frac{c}{b-c} S(n)(n^{\log(b/c)} - 1) & \text{if } b \neq c. \end{cases}$$

If $S(1) > 0$, S and T are non-decreasing, and $S(2n) \leq eS(n)$ for some $e \in \mathbb{R}_{>0}$ and all $n \in \mathbb{N}_{\geq 0}$, then

$$T(n) \in \begin{cases} O(S(n) \log n) & \text{if } b = c, \\ O(S(n) n^{\log(b/c)}) & \text{if } b \neq c. \end{cases}$$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

PROOF. We have the estimate

$$T(2^i) \le b^i T(1) + \sum_{0\le j<i} b^j S(2^{i-j}) \le d2^{i\log b} + S(2^i) \sum_{0\le j<i} \left(\frac{b}{c}\right)^j,$$

where the first inequality follows by induction, and we use $S(2^{i-j}) \le c^{-j} S(2^i)$ in the second one. If $b = c$, then the last sum simplifies to $S(2^i) \cdot i$. If $b \ne c$, then we have a geometric sum

$$\sum_{0\le j<i} \left(\frac{b}{c}\right)^j = \frac{(\frac{b}{c})^i - 1}{\frac{b}{c} - 1} = \frac{c}{b-c}(2^{i(\log(b/c))} - 1),$$

and the first claim follows.

We note that $S(1) \le c^{-i} S(2^i)$ and therefore $d n^{\log b} \le (d/S(1)) \cdot S(n) n^{\log(b/c)}$. Thus

$$T(n) \le \begin{cases} \left(\dfrac{d}{S(1)\log n} + 1\right) S(n)\log n & \text{if } b = c, \\ \left(\dfrac{d}{S(1)} + \dfrac{c}{b-c}\right) S(n) n^{\log(b/c)} & \text{if } b \ne c, \end{cases}$$

for $n = 2^i$ and $i \in \mathbb{N}_{\ge 1}$. For an arbitrary integer $n \ge 2$, let $i = \lceil \log n \rceil$ and $a = d/S(1) + c/(b-c)$. If $b \ne c$, then

$$T(n) \le T(2^i) \le a \cdot S(2^i) 2^{i\log(b/c)} \le a \cdot S(2n) \cdot (2n)^{\log(b/c)} \le \frac{a e b}{c} S(n) n^{\log(b/c)},$$

and similarly for the case $b = c$. □

THEOREM 8.3.

Karatsuba's algorithm 8.1 for multiplying polynomials of degree less than a power n of 2 over a ring can be done with at most $9n^{\log 3}$ or $O(n^{1.59})$ ring operations.

PROOF. In step 3, we have n additions for computing $F_0 + F_1$ and $G_0 + G_1$. The computation of $F_1 G_1 x^n + F_0 G_0$ in step 4 is just a concatenation of coefficients. The computation of $((F_0 + F_1)(G_0 + G_1) - F_0 G_0 - F_1 G_1) x^{n/2}$ takes $2n$ subtractions, and we have another n additions for adding the result to $F_1 G_1 x^n + F_0 G_0$. Thus we may choose $b = 3$, $c = 2$, $d = 1$, and $S(n) = 4n$ in Lemma 8.2 and obtain a total cost of $9 \cdot n^{\log 3} - 8n$ operations. The claim follows from $\log 3 < 1.59$. □

This is a substantial improvement over the classical method, since $\log 3 < 2$. The savings are visualized in Figure 8.2.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

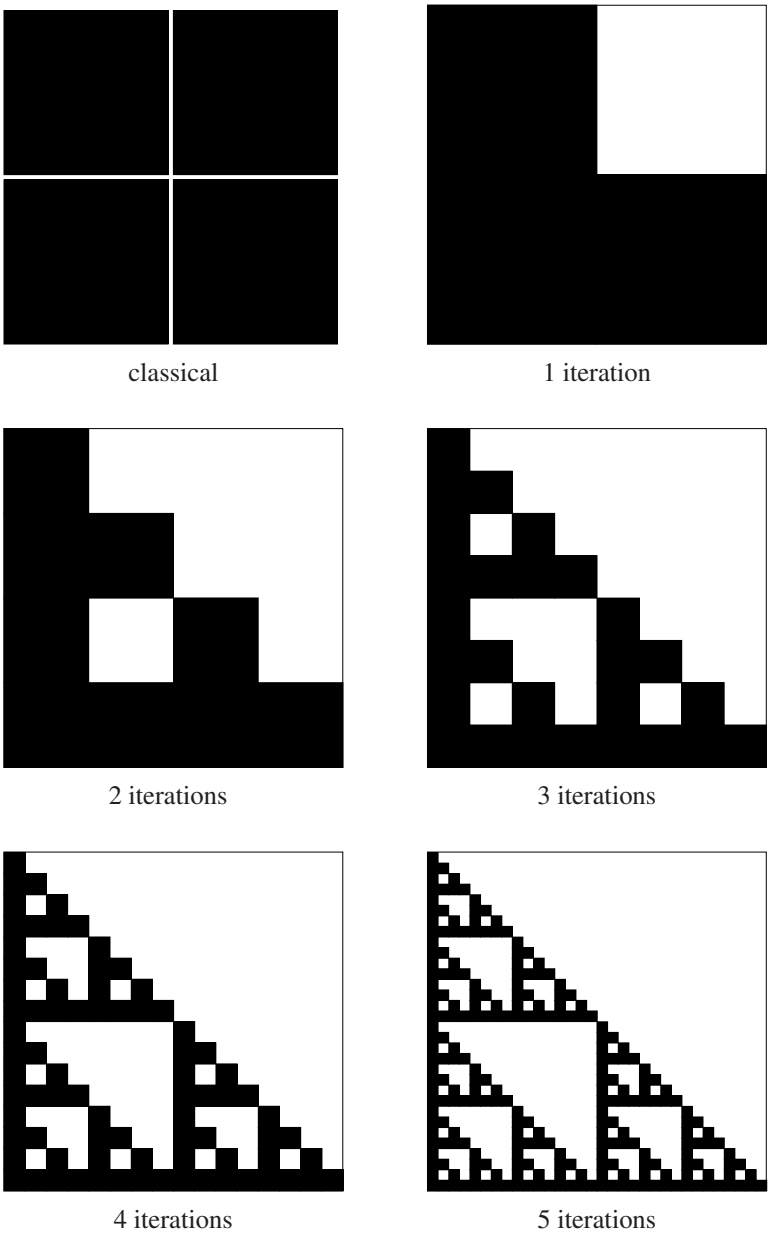


FIGURE 8.2: Cost (= black area) of Karatsuba's algorithm for increasing recursion depths. The image approaches a fractal of dimension $\log_2 3 \approx 1.59$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

If n is not a power of 2, then there are two ways to proceed. The first one is to apply the algorithm for the least power of 2 that is greater than n, that is, for $2^{\lceil \log n \rceil}$. This is easy to analyze, but introduces an additional factor of 3 in the running time, which is annoying if n is only slightly greater than a power of 2. The second possibility is to split the polynomials into blocks of about half the degree each time in the recursive process. This leads to an algorithm that performs better than the first variant, but the analysis is somewhat more involved; see Exercise 8.5.

The same method applies to multiplication of two (positive) integers a and b in r-ary representation, say with $r = 2^{64}$ (see Section 2.1). When they have length at most n, the classical integer multiplication algorithm requires $O(n^2)$ word operations (Section 2.3). Karatsuba's algorithm for integers writes $a = A_1 2^{64m} + A_0$ and $b = B_1 2^{64m} + B_0$, where $A_0, A_1, B_0, B_1 < 2^{64m}$ and $n = 2m$ is assumed to be a power of 2. Then $ab = A_1 B_1 2^{64n} + ((A_0 + A_1)(B_0 + B_1) - A_0 B_0 - A_1 B_1) 2^{64m} + A_0 B_0$. As in the polynomial case, multiplication of two integers has been reduced to multiplication of three integers of at most half the size plus $O(n)$ word operations (if $A_0 + A_1$ or $B_0 + B_1$ exceed 2^{64n}, then one has to take extra care of the leading bits, or alternatively computes $A_0 B_1 + A_1 B_0 = A_0 B_0 + A_1 B_1 - (A_0 - A_1)(B_0 - B_1)$). This leads to the following theorem.

THEOREM 8.4.
Multiplication of two integers of length at most n words can be done with $O(n^{\log 3})$ or $O(n^{1.59})$ word operations using Karatsuba's algorithm.

Karatsuba's algorithm is used in computer algebra systems like MAPLE. The classical method is faster for polynomials of small degree and integers of small length; see Exercises 8.4 and 8.6.

8.2. The Discrete Fourier Transform and the Fast Fourier Transform

In this section, we discuss a polynomial multiplication algorithm which works in nearly linear time. It requires that the coefficient ring contain certain roots of unity. We recall that an element a of a ring R is a **zero divisor** if there exists a nonzero $b \in R$ with $ab = 0$. In particular, 0 is a zero divisor (unless R is the trivial ring $\{0\}$). The reader should note that in many algebra texts, 0 is not considered a zero divisor.

DEFINITION 8.5. *Let R be a ring, $n \in \mathbb{N}_{\geq 1}$, and $\omega \in R$.*

*(i) ω is an **nth root of unity** if $\omega^n = 1$.*

*(ii) ω is a **primitive nth root of unity** (or root of unity of order n) if it is an nth root of unity, $n \in R$ is a unit in R, and $\omega^{n/t} - 1$ is not a zero divisor for any prime divisor t of n.*

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

Here n has two meanings: in "ω^n" it is an integer used as a counter to express the n-fold product of ω with itself, and in "$n \in R$" it stands for the ring element $n \cdot 1_R \in R$, the n-fold sum of 1_R with itself (Section 25.3).

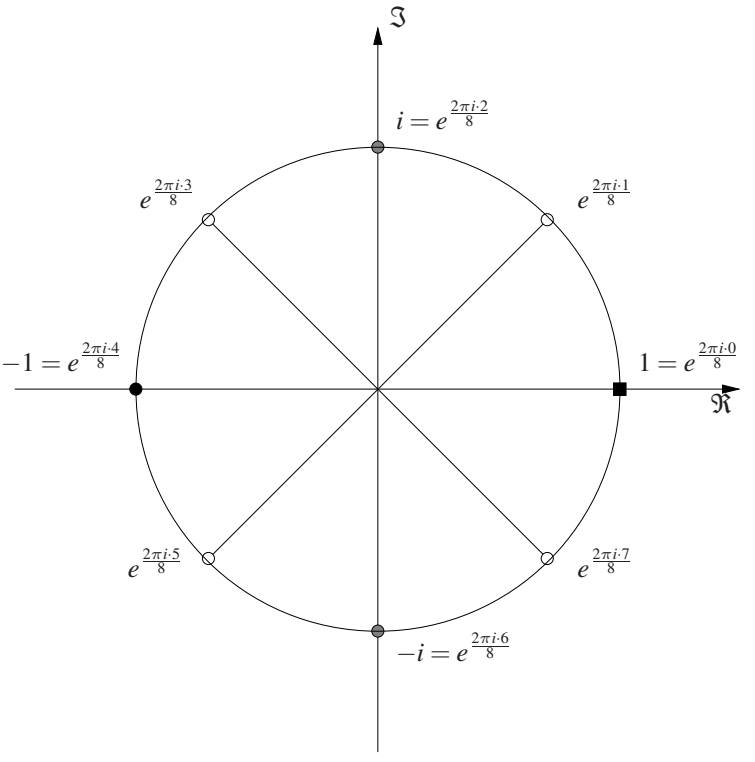


FIGURE 8.3: The 8th roots of unity in $\mathbb{C}$. The black square has order 1, the black circle order 2, the two gray circles order 4, and the four white circles are the primitive 8th roots of unity.

EXAMPLE 8.6. (i) $\omega = e^{2\pi i/8} \in R = \mathbb{C}$ is a primitive 8th root of unity, where $i = \sqrt{-1}$; see Figure 8.3.

(ii) $\mathbb{Z}_8$ has no primitive square root of unity, despite the fact that $3^2 \equiv 1$, since 2 is not a unit.

(iii) For the "Fermat prime" $2^4 + 1 = 17$, the element 3 is a primitive 16th root of unity in $\mathbb{Z}_{17}$, and 2 is not. $\diamond$

The following lemma extends the property of Definition 8.5 (ii) for $\omega^\ell - 1$ from $\ell = n/t$ to all ℓ that are not divisible by n.

LEMMA 8.7. *Let R be a ring, $\ell, n \in \mathbb{N}_{\geq 1}$ such that $1 \leq \ell < n$, and $\omega \in R$ a primitive nth root of unity. Then*

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

(i) $\omega^\ell - 1$ *is not a zero divisor in* R,

(ii) $\sum_{0\le j<n} \omega^{\ell j} = 0$.

PROOF. We will make repeated use of the formula

$$(c-1)\sum_{0\le j<m} c^j = c^m - 1, \tag{1}$$

which holds for all $m \in \mathbb{N}$ and $c \in R$ (in fact, even for an indeterminate c).

(i) Let $g = \gcd(\ell, n)$ and $u, v \in \mathbb{Z}$ so that $u\ell + vn = g$. Since $1 \le g < n$, we can choose a prime divisor t of n so that g divides n/t. Letting $c = \omega^g$ and $m = n/tg$ in (1), we obtain $a \cdot (\omega^g - 1) = \omega^{n/t} - 1$ for some $a \in R$. If $b \in R$ satisfies $b \cdot (\omega^g - 1) = 0$, then also $b \cdot (\omega^{n/t} - 1) = 0$, and hence $b = 0$ since $\omega^{n/t} - 1$ is not a zero divisor. Thus $\omega^g - 1$ is not a zero divisor either.

Furthermore, (1) with $c = \omega^\ell$ and $m = u$ implies that $\omega^\ell - 1$ divides $\omega^{u\ell} - 1 = \omega^{u\ell}\omega^{vn} - 1 = \omega^g - 1$. The same argument as above implies that $\omega^\ell - 1$ is not a zero divisor.

(ii) By letting $c = \omega^\ell$ and $m = n$ in (1), we see that

$$(\omega^\ell - 1)\sum_{0\le j<n} (\omega^\ell)^j = \omega^{\ell n} - 1 = 0.$$

Thus $\sum_{0\le j<n} \omega^{\ell j} = 0$, since $\omega^\ell - 1$ is not a zero divisor, by (i). □

When R is an integral domain (for example, a field), then (i) simply says $\omega^\ell \ne 1$, and it is sufficient to check this for $\ell = n/t$, with t running through the prime divisors of n, for the last property required of a primitive nth root of unity.

The following lemma, proven in Exercise 8.18, says when primitive roots of unity exist in a finite field $\mathbb{F}_q$ (as defined on page 73) with q elements.

LEMMA 8.8. *For a prime power* q *and* $n \in \mathbb{N}$, *a finite field* $\mathbb{F}_q$ *contains a primitive* n*th root of unity if and only if* n *divides* $q-1$.

Let R be a ring, $n \in \mathbb{N}_{\ge 1}$, and $\omega \in R$ be a primitive nth root of unity. In what follows, we identify a polynomial $f = \sum_{0\le j<n} f_j x^j \in R[x]$ of degree less than n with its coefficient vector $(f_0, \dots, f_{n-1}) \in R^n$.

DEFINITION 8.9. *(i) The* R*-linear map*

$$\mathrm{DFT}_\omega\colon \begin{cases} R^n & \longrightarrow & R^n \\ f & \longmapsto & \Big(f(1), f(\omega), f(\omega^2), \dots, f(\omega^{n-1})\Big) \end{cases}$$

which evaluates a polynomial at the powers of ω *is called the* ***Discrete Fourier Transform (DFT)****.*

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

(ii) *The* ***convolution*** *of two polynomials* $f = \sum_{0\le j<n} f_j x^j$ *and* $g = \sum_{0\le k<n} g_k x^k$ *in* $R[x]$ *is the polynomial*

$$h = f *_n g = \sum_{0\le \ell<n} h_\ell x^\ell \in R[x],$$

where

$$h_\ell = \sum_{j+k\equiv \ell \bmod n} f_j g_k = \sum_{0\le j<n} f_j g_{\ell-j} \text{ for } 0 \le \ell < n,$$

with index arithmetic modulo n*. If* n *is clear from the context, we will simply write* $*$ *for* $*_n$*. If we regard the coefficients as vectors in* R^n*, then* h *is called the* ***cyclic convolution*** *of the vectors* f *and* g*.*

This notion of convolution is equivalent to polynomial multiplication in the ring $R[x]/\langle x^n - 1\rangle$. The ℓth coefficient of the product polynomial fg is $\sum_{j+k\equiv \ell \bmod n} f_j g_k$, and hence $f *_n g \equiv fg \bmod x^n - 1$. We will exploit this relationship between convolution and multiplication to obtain fast polynomial multiplication algorithms.

EXAMPLE 8.10. Let $R = \mathbb{Q}$, $n = 4$, $u \in \mathbb{Q}$, and suppose we want to multiply $f = x^3 + 1$ and $g = 2x^3 + 3x^2 + x + 1$ in $\mathbb{Q}[x]$ modulo $x^4 - u$. Then

$$\begin{aligned} fg &= 2x^6 + 3x^5 + x^4 + 3x^3 + 3x^2 + x + 1 \\ &= (2x^2 + 3x + 1)(x^4 - u) + 3x^3 + 3x^2 + x + 1 + (2x^2 + 3x + 1)u \\ &= (2x^2 + 3x + 1)(x^4 - u) + 3x^3 + (3 + 2u)x^2 + (1 + 3u)x + (1 + u). \end{aligned}$$

The special form of the binomial $x^4 - u$ makes division with remainder particularly easy: the quotient is the upper part of fg, and the remainder is the lower part of fg plus u times the upper part. In particular, for $u = 1$ we have $fg \equiv 3x^3 + 5x^2 + 4x + 2 \bmod x^4 - 1$, or equivalently, $f *_4 g = 3x^3 + 5x^2 + 4x + 2$. A similar phenomenon will help us in the FFT algorithm below. $\diamond$

LEMMA 8.11. *For polynomials* $f, g \in R[x]$ *of degree less than* n*, we have*

$$\mathrm{DFT}_\omega(f * g) = \mathrm{DFT}_\omega(f) \cdot \mathrm{DFT}_\omega(g),$$

where $\cdot$ *denotes the pointwise multiplication of vectors.*

PROOF. We have $f * g = fg + q \cdot (x^n - 1)$ for some $q \in R[x]$, so that

$$(f * g)(\omega^j) = f(\omega^j)g(\omega^j) + q(\omega^j)(\omega^{jn} - 1) = f(\omega^j)g(\omega^j) \text{ for } 0 \le j < n. \ \square$$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

We may also consider the map $R[x] \longrightarrow R^n$ that evaluates f at $\omega^0, \ldots, \omega^{n-1}$. Its kernel is $\langle x^n - 1 \rangle$, and the lemma says that $\mathrm{DFT}_\omega : R[x]/\langle x^n - 1\rangle \longrightarrow R^n$ is a homomorphism of R-algebras, where multiplication in R^n is pointwise multiplication of vectors. The following commutative diagram illustrates this:

$$\begin{array}{ccc} \left(R[x]/\langle x^n-1\rangle\right)^2 & \xrightarrow{\mathrm{DFT}_\omega \times \mathrm{DFT}_\omega} & R^n \times R^n \\ \Big\downarrow \text{cyclic convolution} & & \Big\downarrow \text{pointwise multiplication} \\ R[x]/\langle x^n-1\rangle & \xrightarrow[\mathrm{DFT}_\omega]{} & R^n \end{array} \tag{2}$$

In fact, DFT_ω is an isomorphism. If R is a field, then this is the special case of the Chinese Remainder Theorem 5.3, where $m_j = x - \omega^j$ for $0 \le j < n$. We discussed in Section 5.1 the general principle of **change of representation** and will now see how the particular example (2) gives rise to a fast multiplication algorithm.

In the following, polynomials of degree less than n over an integral domain R are—besides the usual **dense** representation by their coefficient vectors—represented by their values at n distinct points $u_0, \ldots, u_{n-1} \in R$, namely the powers $u_j = \omega^j$ for $0 \le j < n$ of a primitive nth root of unity $\omega \in R$. The reason for considering the value representation is that multiplication in that representation is easy: If $f(u_0), \ldots, f(u_{n-1})$ and $g(u_0), \ldots, g(u_{n-1})$ are the values of two polynomials f and g with $\deg(fg) < n$ at n distinct points, then the values of the product polynomial fg at those points are $f(u_0) \cdot g(u_0), \ldots, f(u_{n-1}) \cdot g(u_{n-1})$. Hence the cost of polynomial multiplication in the value representation is linear in the degree, while we do not know how to multiply polynomials in the dense representation in linear time. Thus a fast way of doing multipoint evaluation and interpolation leads to a fast polynomial multiplication algorithm: evaluate the two input polynomials, multiply the results pointwise, and finally interpolate to get the product polynomial.

The Discrete Fourier Transform is a special multipoint evaluation at the powers $1, \omega, \ldots, \omega^{n-1}$ of a primitive nth root of unity ω, and we will now show that both the DFT and its inverse, the interpolation at the powers of ω, can be computed with $O(n \log n)$ operations in R, and thus obtain an $O(n \log n)$ multiplication algorithm for polynomials. In Chapter 10, we will see a fast algorithm for evaluation and interpolation at arbitrary points.

First we show that interpolation at the powers of ω is essentially again a Discrete Fourier Transform. The Vandermonde matrix

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

$$V_\omega = \mathrm{VDM}(1,\omega,\dots,\omega^{n-1}) = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)^2} \end{pmatrix}$$
$$= (\omega^{jk})_{0\le j,k<n} \in R^{n\times n}$$

is the matrix of the multipoint evaluation map DFT_ω (Section 5.2).

EXAMPLE 8.12. For the primitive 4th root of unity $\omega = i = \sqrt{-1} \in \mathbb{C}$, we have

$$V_i = \mathrm{VDM}(1,i,-1,-i) = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}. \diamond$$

Since $1,\omega,\dots,\omega^{n-1}$ are distinct, V_ω is invertible if R is a field (Section 5.2), and its inverse is the matrix of the interpolation map at the n points. The following theorem says that this is true for arbitrary rings.

THEOREM 8.13.
Let R be a ring (commutative, with 1), $n \in \mathbb{N}_{\ge 1}$, and $\omega \in R$ be a primitive nth root of unity. Then ω^{-1} is a primitive nth root of unity and $V_\omega \cdot V_{\omega^{-1}} = nI$, where I is the $n \times n$ identity matrix.

PROOF. Exercise 8.13 shows that ω^{-1} is a primitive nth root of unity. Let $0 \le j,\ell < n$, and

$$u = (V_\omega \cdot V_{\omega^{-1}})_{j\ell} = \sum_{0\le k<n} (V_\omega)_{jk}(V_{\omega^{-1}})_{k\ell} = \sum_{0\le k<n} \omega^{jk}\omega^{-k\ell} = \sum_{0\le k<n} (\omega^{j-\ell})^k.$$

If $j = \ell$, then $u = \sum_{0\le k<n} 1 = n$. If $j \ne \ell$, then $u = \sum_{0\le k<n} \omega^{(j-\ell)k} = 0$, by Lemma 8.7. □

In particular, the theorem implies that $(V_\omega)^{-1} = n^{-1}V_{\omega^{-1}}$, so that computing the inverse is fairly easy.

EXAMPLE 8.12 (continued). $V_i^{-1} = \frac{1}{4}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{pmatrix}. \diamond$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

We next discuss an important algorithm—the **Fast Fourier Transform**, or FFT for short—that computes the DFT quickly. It was (re)discovered by Cooley and Tukey in 1965, and is possibly the second most important nontrivial algorithm in practice. (Contender for first place is fast sorting.) The inverse DFT can then also be computed quickly, by Theorem 8.13.

Let $n \in \mathbb{N}_{\geq 1}$ be even, $\omega \in R$ a primitive nth root of unity, and $f \in R[x]$ of degree less than n. To evaluate f at the powers $1, \omega, \omega^2, \ldots, \omega^{n-1}$, we divide f by $x^{n/2}-1$ and $x^{n/2}+1$ with remainder:

$$f = q_0(x^{n/2}-1) + r_0 = q_1(x^{n/2}+1) + r_1, \tag{3}$$

for some $q_0, r_0, q_1, r_1 \in R[x]$ of degree less than $n/2$. Due to the special form of the divisor polynomials, the computation of the remainders r_0 and r_1 (we do not actually need the quotients) can be done by adding the upper $n/2$ coefficients of f to, respectively subtracting them from, the lower $n/2$ coefficients, as in Example 8.10, at a total cost of n operations in R. In other words, if $f = F_1 x^{n/2} + F_0$ with $\deg F_0, \deg F_1 < n/2$, then $x^{n/2}-1$ divides $f - F_0 - F_1$, and hence $r_0 = F_0 + F_1$; similarly $r_1 = F_0 - F_1$. If we plug in a power of ω for x in (3), we find

$$\begin{aligned} f(\omega^{2\ell}) &= q_0(\omega^{2\ell})(\omega^{n\ell}-1) + r_0(\omega^{2\ell}) = r_0(\omega^{2\ell}), \\ f(\omega^{2\ell+1}) &= q_1(\omega^{2\ell+1})(\omega^{n\ell}\omega^{n/2}+1) + r_1(\omega^{2\ell+1}) = r_1(\omega^{2\ell+1}) \end{aligned}$$

for all $0 \leq \ell < n/2$. We have used the facts that $\omega^{n\ell} = 1$ and $\omega^{n/2} = -1$, since

$$0 = \omega^n - 1 = (\omega^{n/2}-1)(\omega^{n/2}+1)$$

and $\omega^{n/2}-1$ is not a zero divisor. It remains to evaluate r_0 at the even powers of ω and r_1 at the odd powers. Now ω^2 is a primitive $(n/2)$th root of unity (Exercise 8.13), and hence the first task is a DFT of order $n/2$. But also the evaluation of r_1 can be reduced to a DFT of order $n/2$ by noting that $r_1(\omega^{2\ell+1}) = r_1^*(\omega^{2\ell})$ for $r_1^* = r_1(\omega x)$. The computation of the coefficients of r_1^* uses $n/2$ multiplications by powers of ω. If n is a power of 2, we can proceed recursively to evaluate r_0 and r_1^* at the powers $1, \omega^2, \ldots, \omega^{2n-2}$ of ω^2, and obtain the following algorithm.

ALGORITHM 8.14 Fast Fourier Transform (FFT).
Input: $n = 2^k \in \mathbb{N}$ with $k \in \mathbb{N}$, $f = \sum_{0 \leq j < n} f_j x^j \in R[x]$, and the powers $\omega, \omega^2, \ldots, \omega^{n-1}$ of a primitive nth root of unity $\omega \in R$.
Output: $\mathrm{DFT}_\omega(f) = (f(1), f(\omega), \ldots, f(\omega^{n-1})) \in R^n$.

1. **if** $n = 1$ **then return** (f_0)
2. $r_0 \longleftarrow \sum_{0 \leq j < n/2} (f_j + f_{j+n/2}) x^j, \quad r_1^* \longleftarrow \sum_{0 \leq j < n/2} (f_j - f_{j+n/2}) \omega^j x^j$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

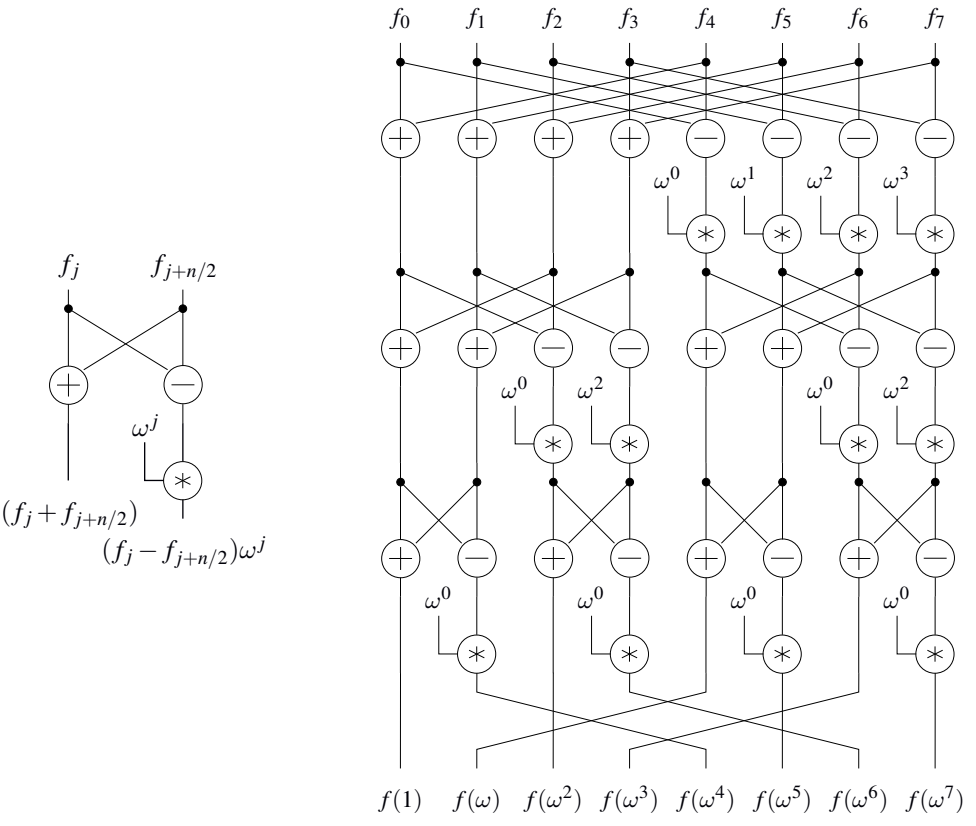


FIGURE 8.4: A butterfly operation (left) and an arithmetic circuit computing the FFT for $n = 8$ (right). A subtraction node computes the difference of its left input minus its right input.

3. **call** the algorithm recursively to evaluate r_0 and r_1^* at the powers of ω^2

4. **return** $\left(r_0(1), r_1^*(1), r_0(\omega^2), r_1^*(\omega^2), \ldots, r_0(\omega^{n-2}), r_1^*(\omega^{n-2})\right)$

THEOREM 8.15.
Let n be a power of 2 and $\omega \in R$ be a primitive nth root of unity. Then Algorithm 8.14 correctly computes DFT_ω *using $n \log n$ additions in R and $(n/2)\log n$ multiplications by powers of ω, in total $\frac{3}{2} n \log n$ ring operations.*

PROOF. We prove correctness by induction on k. If $k = 0$, then $f = f_0$ is constant and the algorithm returns the correct result. If $k > 1$, we have to show that $f(\omega^{2\ell}) = r_0(\omega^{2\ell})$ and $f(\omega^{2\ell+1}) = r_1^*(\omega^{2\ell})$ for all $0 \le \ell < n/2$, and then the claim follows from

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

the induction hypothesis. Retracing the calculations above, we find

$$r_0(\omega^{2\ell}) = \sum_{0\le j<n/2} (f_j + f_{j+n/2})\omega^{2\ell j} = \sum_{0\le j<n/2} f_j\omega^{2\ell j} + \sum_{0\le j<n/2} f_{j+n/2}\omega^{2\ell j}\omega^{\ell n}$$
$$= \sum_{0\le j<n} f_j\omega^{2\ell j} = f(\omega^{2\ell}),$$

$$r_1^*(\omega^{2\ell}) = \sum_{0\le j<n/2} (f_j - f_{j+n/2})\omega^j\omega^{2\ell j}$$
$$= \sum_{0\le j<n/2} f_j\omega^{(2\ell+1)j} + \sum_{0\le j<n/2} f_{j+n/2}\omega^{(2\ell+1)j}\omega^{\ell n}\omega^{n/2}$$
$$= \sum_{0\le j<n} f_j\omega^{(2\ell+1)j} = f(\omega^{2\ell+1}).$$

Let $S(n)$ and $T(n)$ denote the number of additions and multiplications in R, respectively, that the algorithm uses for input size n. The cost for the individual steps is: 0 in steps 1 and 4, n additions and $n/2$ multiplications in step 2, and $2S(n/2)$ additions and $2T(n/2)$ multiplications in step 3. This yields $S(1) = T(1) = 0$, $S(n) = 2S(n/2) + n$, and $T(n) = 2T(n/2) + n/2$, and by unfolding the recursions we find that $S(n) = n\log n$ and $T(n) = \frac{1}{2}n\log n$. □

The FFT can be nicely illustrated in form of an arithmetic circuit. It is built from elementary blocks that execute step 2 of the above algorithm for one particular value of j, called a **butterfly operation**. One such building block as well as the entire circuit for $n = 8$ are shown in Figure 8.4.

Figure 8.5 illustrates the cost of Algorithm 8.14 if the recursion is stopped at depths $0, 1, \ldots, 5$ and the remaining subproblems are computed by Horner's rule. The diagonal lines visualize the linear cost at each recursive step which—in contrast to Karatsuba's method—contributes to the overall cost. Now we use the FFT to compute convolutions and products of polynomials quickly.

ALGORITHM 8.16 Fast convolution.
Input: $f, g \in R[x]$ of degree less than $n = 2^k$ with $k \in \mathbb{N}$, and a primitive nth root of unity $\omega \in R$.
Output: $f * g \in R[x]$.

1. compute $\omega^2, \ldots, \omega^{n-1}$
2. $\alpha \longleftarrow \mathrm{DFT}_\omega(f), \quad \beta \longleftarrow \mathrm{DFT}_\omega(g)$
3. $\gamma \longleftarrow \alpha \cdot \beta$ { pointwise product }
4. **return** $\mathrm{DFT}_\omega{}^{-1}(\gamma) = \dfrac{1}{n}\mathrm{DFT}_{\omega^{-1}}(\gamma)$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

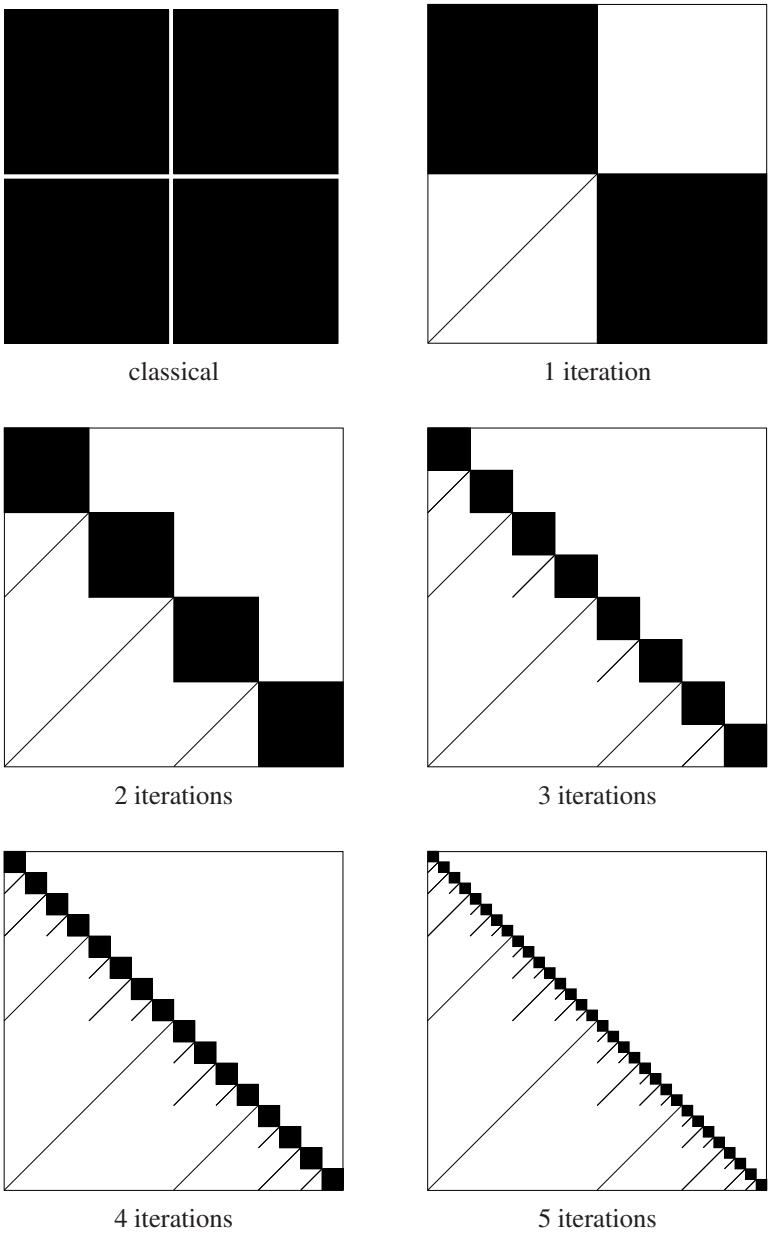


FIGURE 8.5: Cost of the FFT for increasing recursion depths. The black area is proportional to the total work.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

DEFINITION 8.17. *We say that a (commutative) ring R* ***supports the FFT*** *if R has a primitive 2^kth root of unity for any $k \in \mathbb{N}$.*

An example of a ring supporting the FFT is $R = \mathbb{C}$.

THEOREM 8.18.
Let R be a ring that supports the FFT, and $n = 2^k$ for some $k \in \mathbb{N}$. Then convolution in $R[x]/\langle x^n - 1\rangle$ and multiplication of polynomials $f, g \in R[x]$ with $\deg(fg) < n$ can be performed using $3n\log n$ additions in R, $\frac{3}{2}n\log n + n - 2$ multiplications by powers of ω, n multiplications in R, and n divisions by n, in total $\frac{9}{2}n\log n + O(n)$ arithmetic operations.

PROOF. Let $f, g \in R[x]$ of degree less than n and $h = f * g$. Then h is uniquely determined by its values at n distinct points. Since the convolution satisfies

$$\mathrm{DFT}_\omega(h) = \mathrm{DFT}_\omega(f) \cdot \mathrm{DFT}_\omega(g),$$

by Lemma 8.11, where the multiplication is the componentwise product, we conclude that Algorithm 8.16 correctly computes the convolution of f and g. In particular, the output does not depend on the choice of the primitive root of unity ω. If furthermore $\deg(fg) < n$, then $f * g \equiv fg \bmod (x^n - 1)$ implies that $fg = f * g$.

The cost for the individual steps is

1. $n - 2$ multiplications by ω,
2. $2n\log n$ additions and $n\log n$ multiplications by powers of ω,
3. n multiplications,
4. $n\log n$ additions, $\frac{1}{2}n\log n$ multiplications by powers of ω, and n divisions by n,

and the claim follows. □

To multiply two arbitrary polynomials of degree less than $n \in \mathbb{N}$, we only need a primitive 2^kth root of unity, where $2^{k-1} < 2n \leq 2^k$. Then we have decreased the cost of about $2n^2$ of the classical algorithm to $O(n\log n)$.

COROLLARY 8.19.
If R supports the FFT, then polynomials in $R[x]$ of degree less than n can be multiplied with $18n\log n + O(n)$ operations in R.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

8.3. Schönhage and Strassen's multiplication algorithm

In the previous section we have discussed an asymptotically fast FFT-based polynomial multiplication algorithm over rings containing certain primitive roots of unity. If the underlying coefficient ring R is arbitrary, an FFT-based approach will not work directly. In this section, we will show how to make it work by adjoining "virtual" roots of unity.

Let R be a ring such that 2 is a unit in R, $n = 2^k$ for some $k \in \mathbb{N}$, and $D = R[x]/\langle x^n + 1\rangle$. D is generally not a field, even if R is, as the example $R = \mathbb{C}$ and $n = 2$ shows. The congruences

$$x^n \equiv -1 \bmod (x^n + 1), \quad x^{2n} = (x^n)^2 \equiv 1 \bmod (x^n + 1)$$

imply that $\omega = x \bmod (x^n + 1) \in D$ is a $2n$th root of unity. Moreover, $\omega^n - 1 = (-1) - 1 = -2$ is a unit in R since 2 is, and ω is a primitive $2n$th root of unity.

To multiply two polynomials $f, g \in R[x]$ with $\deg(fg) < n = 2^k \in \mathbb{N}$, it is clearly sufficient to compute fg modulo $x^n + 1$. This is called the **negative wrapped convolution of f and g**. We let $m = 2^{\lfloor k/2 \rfloor}$, $t = n/m = 2^{\lceil k/2 \rceil}$, and partition the coefficients of f and g into t blocks of size m, that is, we write

$$f = \sum_{0 \le j < t} f_j x^{mj}, \quad g = \sum_{0 \le j < t} g_j x^{mj},$$

with $f_j, g_j \in R[x]$ of degree less than m for $0 \le j < t$. With $f' = \sum_{0 \le j < t} f_j y^j, g' = \sum_{0 \le j < t} g_j y^j \in R[x,y]$, we then have $f = f'(x, x^m)$ and $g = g'(x, x^m)$. It is sufficient to compute $f'g'$ modulo $y^t + 1$, since

$$f'g' = h' + q'(y^t + 1) \equiv h' \bmod (y^t + 1) \tag{4}$$

for some $h', q' \in R[x,y]$ implies that

$$fg = h'(x, x^m) + q'(x, x^m)(x^{tm} + 1) \equiv h'(x, x^m) \bmod (x^n + 1).$$

We now take the primitive $4m$th root of unity

$$\xi = x \bmod (x^{2m} + 1) \in D = R[x]/\langle x^{2m} + 1\rangle.$$

We want to compute $h' \in R[x,y]$ with $\deg_y h' < t$ satisfying (4) (this uniquely determines h'; see Section 2.4). Comparing coefficients of y^j for $j \ge t$, we see that $\deg_x q' \le \deg_x(f'g') < 2m$ and conclude that

$$\deg_x h' \le \max\{\deg_x(f'g'), \deg_x q'\} < 2m. \tag{5}$$

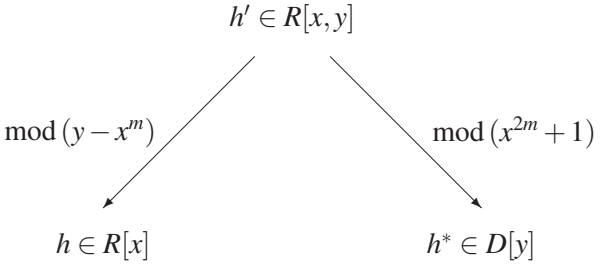
With $f^* = f' \bmod (x^{2m} + 1)$, $g^* = g' \bmod (x^{2m} + 1)$, and $h^* = h' \bmod (x^{2m} + 1)$ in $D[y]$, (4) implies

$$f^*g^* \equiv h^* \bmod (y^t + 1) \text{ in } D[y]. \tag{6}$$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

Since the three polynomials have degrees in x less than $2m$, by (5), reducing them modulo $x^{2m}+1$ is just taking a different algebraic meaning of the same coefficient array. In particular, the coefficients of $h' \in R[x][y]$ can be read off the coefficients of $h^* \in D[y]$.

The following picture illustrates the relations between h, h' and h^*; the arrows are ring homomorphisms.



For example, let $h = 4x^3+3x^2+2x+1 \in \mathbb{Q}[x]$, $m=2$, and $\xi = x \bmod (x^4+1) \in D = \mathbb{Q}[x]/\langle x^4+1\rangle$. Then $h' = (4x+3)y+(2x+1) \in \mathbb{Q}[x,y]$ and $h^* = (4\xi+3)y+(2\xi+1) \in D[y]$.

Computationally, "nothing happens" in mapping h' to h^*, but we are now in a situation where we can apply the machinery of the FFT to compute (6), as follows. Since t equals either m or $2m$, D contains a primitive $2t$th root of unity η, namely $\eta = \xi$ if $t = 2m$ and $\eta = \xi^2$ if $t = m$. Then (6) is equivalent to

$$f^*(\eta y)g^*(\eta y) \equiv h^*(\eta y) \bmod \Big((\eta y)^t + 1\Big),$$

or

$$f^*(\eta y)g^*(\eta y) \equiv h^*(\eta y) \bmod (y^t - 1), \tag{7}$$

since $\eta^t = -1$. Given $f^*(\eta y)$ and $g^*(\eta y)$ in $D[y]$, Algorithm 8.16 computes $h^*(\eta y)$ with $O(t \log t)$ operations in D, using essentially three t-point FFTs. A multiplication of two elements in D is again a negative wrapped convolution over R which can be handled recursively. Putting things together, we obtain the following algorithm.

ALGORITHM 8.20 Fast negative wrapped convolution.

Input: Two polynomials $f, g \in R[x]$ of degree less than $n = 2^k$ for some $k \in \mathbb{N}$, where R is a (commutative) ring such that 2 is a unit in R.

Output: $h \in R[x]$ such that $fg \equiv h \bmod (x^n+1)$ and $\deg h < n$.

1. **if** $k \leq 2$ **then**
   call the classical algorithm 2.3 (or Karatsuba's algorithm 8.1) to compute $f \cdot g$
   return $fg \operatorname{rem} x^n+1$

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

2. $m \longleftarrow 2^{\lfloor k/2 \rfloor}, \quad t \longleftarrow n/m$
   let $f', g' \in R[x,y]$ with $\deg_x f', \deg_x g' < m$ such that $f = f'(x, x^m)$ and $g = g'(x, x^m)$

3. let $D = R[x]/\langle x^{2m}+1 \rangle$
   if $t = 2m$ **then** $\eta \longleftarrow x \bmod (x^{2m}+1)$ **else** $\eta \longleftarrow x^2 \bmod (x^{2m}+1)$
   { η is a primitive $2t$th root of unity }
   $f^* \longleftarrow f' \bmod (x^{2m}+1), \quad g^* \longleftarrow g' \bmod (x^{2m}+1)$
   call the fast convolution algorithm 8.16 with $\omega = \eta^2$ to compute $h^* \in D[y]$ of degree less than t such that

$$f^*(\eta y) g^*(\eta y) \equiv h^*(\eta y) \bmod (y^t - 1),$$

   using Algorithm 8.20 recursively for the multiplications in D

4. let $h' \in R[x,y]$ with $\deg_x h' < 2m$ such that $h^* = h' \bmod (x^{2m}+1)$
   $h \longleftarrow h'(x, x^m) \operatorname{rem} (x^n+1)$
   return h

EXAMPLE 8.21. Let $R = \mathbb{F}_5$, $f = x^4 + 2x + 3$, and $g = 2x^3 + x^2 + 4x + 2$ in $\mathbb{F}_5[x]$. Then we can compute $fg \in \mathbb{F}_5[x]$ using Algorithm 8.20 with $k = 3$, as follows. In step 2, we calculate $m = 2$, $t = 4$, $f' = y^2 + 2x + 3$ and $g' = (2x+1)y + 4x + 2$. In step 3, we have $\eta = x \bmod (x^4+1)$,

$$\begin{aligned} f^* &= y^2 + 2\eta + 3, & g^* &= (2\eta+1)y + 4\eta + 2, \\ f^*(\eta y) &= \eta^2 y^2 + 2\eta + 3, & g^*(\eta y) &= (2\eta^2 + \eta)y + 4\eta + 2. \end{aligned}$$

Now Algorithm 8.16 (or, in this case, a direct calculation of $f^*(\eta y) g^*(\eta y)$) yields

$$\begin{aligned} h^*(\eta y) &= (2\eta^4 + \eta^3)y^3 + (4\eta^3 + 2\eta^2)y^2 + (4\eta^3 + 3\eta^2 + 3\eta)y + 3\eta^2 + \eta + 1, \\ h^* &= h^*(\eta(\eta^{-1}y)) \\ &= (2\eta+1)y^3 + (4\eta+2)y^2 + (4\eta^2 + 3\eta + 3)y + 3\eta^2 + \eta + 1. \end{aligned}$$

Finally, in step 4 we have

$$\begin{aligned} h' &= (2x+1)y^3 + (4x+2)y^2 + (4x^2 + 3x + 3)y + (3x^2 + x + 1), \\ h'(x, x^2) &= 2x^7 + x^6 + 4x^5 + x^4 + 3x^3 + x^2 + x + 1 \equiv fg \bmod (x^8+1), \end{aligned}$$

and in fact equality holds in the second line since the degrees of both sides are less than 8. ◇

THEOREM 8.22.
Algorithm 8.20 works correctly and uses $\frac{9}{2} n \log n \log\log n + O(n \log n)$ *operations in* R.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

PROOF. Correctness follows from the discussion preceding the algorithm. Let $T(k)$ denote the number of arithmetic operations in R that the algorithm uses on inputs of size $n = 2^k$. The cost for step 1 is $O(1)$. In step 2, no arithmetic operations in R are performed. By Theorem 8.18, Algorithm 8.16 uses $3t \log t$ additions in D and $\frac{3}{2}t \log t$ multiplications by powers of $\omega = \eta^2$ in the FFT-steps, plus t divisions by $t \in R$ and t "essential" multiplications of two arbitrary elements of D (the powers of ω in step 1 of Algorithm 8.16 need not be computed). One addition in D costs $2m$ additions in R, one division by t costs $2m$ divisions in R, and one multiplication of $a = \sum_{0 \le j < 2m} a_j x^j \bmod (x^{2m}+1)$ by a power of η corresponds to a cyclic shift of the coordinates a_i and a sign inversion of the "wrapped around" coordinates, using at most $2m$ operations in R. Each essential multiplication in D is done recursively, using $T(\lfloor k/2 \rfloor + 1)$ operations in R. The computation of $f^*(\eta y), g^*(\eta y)$ from f^*, g^* and of $h^* = h^*(\eta(\eta^{-1}y))$ from $h^*(\eta y)$ amounts to $3t$ multiplications by powers of η. Thus the total cost of step 3 is at most $9mt \log t + 8mt + t \cdot T(\lfloor k/2 \rfloor + 1)$. In step 4, the only cost is at most $n = mt$ additions for the computation of h from h'. Together, we have

$$T(k) \le 2^{\lceil k/2 \rceil} T\left(\left\lfloor \frac{k}{2} \right\rfloor + 1\right) + 9 \cdot 2^k \left(\left\lceil \frac{k}{2} \right\rceil + 1\right)$$

if $k > 2$. Thus

$$\begin{aligned} 2^{-k} T(k+1) + 45 &\le 2^{\lceil (k+1)/2 \rceil - k} T\left(\left\lfloor \frac{k+1}{2} \right\rfloor + 1\right) + 90 + 18\left(\left\lceil \frac{k+1}{2} \right\rceil + 1\right) - 45 \\ &= 2\left(2^{-\lceil k/2 \rceil} T\left(\left\lceil \frac{k}{2} \right\rceil + 1\right) + 45\right) + 18\left(\left\lfloor \frac{k}{2} \right\rfloor - \frac{1}{2}\right) \end{aligned}$$

if $k > 1$, where we used $\lfloor (k+1)/2 \rfloor = \lceil k/2 \rceil$ and $\lceil (k+1)/2 \rceil = \lfloor k/2 \rfloor + 1$. Writing $S(k) = (2^{-k} T(k+1) + 45)/(k-1)$, we obtain

$$\begin{aligned} S(k) &\le \frac{2(\lceil k/2 \rceil - 1)}{k-1} S\left(\left\lceil \frac{k}{2} \right\rceil\right) + 9 \frac{2(\lfloor k/2 \rfloor - 1/2)}{k-1} \\ &\le S\left(\left\lceil \frac{k}{2} \right\rceil\right) + 9 \le \cdots \le S(2) + 9(\lceil \log k \rceil - 1) \end{aligned}$$

for $k > 1$, by induction, and hence

$$\begin{aligned} T(k) &= 2^{k-1}\Big((k-2)S(k-1) - 45\Big) \\ &\le \frac{9}{2} 2^k (k-2)(\lceil \log(k-1) \rceil - 1) + \frac{S(2)}{2} 2^k (k-2) - \frac{45}{2} 2^k \\ &\in \frac{9}{2} 2^k k \log k + O(2^k k) = \frac{9}{2} n \log n \log\log n + O(n \log n). \ \square \end{aligned}$$

In an implementation, one has to carefully determine the "crossover" threshold for k in step 1 below which one uses a different algorithm.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

We note that the value of m in step 2 and also the value of n need not be powers of 2; it is sufficient that t be a power of 2 dividing $2m$. This allows for choices of n such as $n = 3 \cdot 2^k$ or $n = 5 \cdot 3^k$, which have the advantage that fewer zeroes have to be padded when $\deg(fg)$ is just below such a number. For example, if $\deg(fg) = 3 \cdot 2^{2l-1} - 1$ for some $l \in \mathbb{N}$, then the literal approach would use $n = 2^{2l+1}$ and $m = 2^l$, $t = 2^{l+1}$ in step 2, while it seems better to choose $n = 3 \cdot 2^{2l-1}$, with $m = 3 \cdot 2^{l-1}$ and $t = 2^l$ in step 2.

Exercise 8.30 discusses the analog of Algorithm 8.20 when 3 is a unit in R, using a 3-adic FFT. In particular, this covers the case when R is a field of characteristic 2. What about arbitrary rings R? All divisions in Algorithm 8.16 are by powers of two. We replace the last line by

4. **return** $n \cdot \mathrm{DFT}_\omega{}^{-1}(\gamma) = \mathrm{DFT}_{\omega^{-1}}(\gamma)$

Then it uses only additions and multiplications, but no divisions in R, and returns $n \cdot (a * b)$ instead of $a * b$. If we use this modified algorithm in step 3 of Algorithm 8.20, then Exercise 8.31 shows that the modified algorithm works over any (commutative) ring and returns $2^\kappa \cdot h$ for some $\kappa \in \mathbb{N}$. Similar modifications of the 3-adic FFT algorithm from Exercise 8.30 lead to an algorithm that computes $3^\lambda \cdot h$ for some $\lambda \in \mathbb{N}$.

If we now want to compute the product of two polynomials $f, g \in R[x]$ of degree less than n, we choose $k, l \in \mathbb{N}$ such that $2^{k-2} < n \leq 2^{k-1}$ and $3^{l-1} < n \leq 3^l$ and call both the modified 2-adic and the modified 3-adic algorithm to compute $2^\kappa fg$ and $3^\lambda fg$. Using the Extended Euclidean Algorithm, we find $s, t \in \mathbb{Z}$ such that $s2^\kappa + t3^\lambda = 1$, and obtain $s2^\kappa fg + t3^\lambda fg = fg$. Thus we have the following result.

THEOREM 8.23.
Over any commutative ring R, polynomials of degree less than n can be multiplied using at most $(18 + 72\log_3 2)\, n \log n \log\log n + O(n \log n)$ or $63.43\, n \log n \log\log n + O(n \log n)$ arithmetic operations in R.

PROOF. Let $k, l \in \mathbb{N}$ be as above. By Theorem 8.22, computing $2^\kappa fg$ modulo $x^{2^k} + 1$ takes $\frac{9}{2} 2^k k \log k + O(k \log k)$ or $18\, n \log n \log\log n + O(n \log n)$ operations, since $2^k < 4n$. Similarly, $3^\lambda fg$ modulo $x^{2 \cdot 3^k} + x^{3^k} + 1$ can be computed using $24 \cdot 3^k k \log k + O(k \log k)$ or $72\, n \log_3 n \log\log n + O(n \log n)$ operations, by Exercise 8.30 and the fact that $3^k < 3n$. Computing $s2^\kappa fg + t3^\lambda fg$ takes at most $6n$ operations, and the claim follows by noting that $\log_3 n = \log_3 2 \cdot \log n$ and $72 \log_3 2 < 45.43$. □

The constant 63.43 is probably not the best possible, and the reader is encouraged to figure out a smaller constant herself.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

Can this method be extended to integer multiplication? The following result introduced the FFT into algebraic complexity theory and high-performance computing.

THEOREM 8.24 *Schönhage & Strassen (1971).* *Multiplication of integers of length n can be performed with $O(n \log n \operatorname{loglog} n)$ word operations.*

We do not present the details of this algorithm (it is partly described in Exercise 8.36), but rather a different approach for fast integer multiplication which only works for integers of bounded length but seems sufficient for most practical purposes (inputs up to millions of gigabytes). Let $a = \sum_{0 \le j < n} a_j 2^{64j}$, $b = \sum_{0 \le k < n} b_k 2^{64k}$ in 2^{64}-ary representation, and $A = \sum_{0 \le j < n} a_j x^j$, $B = \sum_{0 \le k < n} b_k x^k$ in $\mathbb{Z}[x]$, so that $a = A(2^{64})$ and $b = B(2^{64})$. If $C = AB = \sum_{0 \le l < 2n-1} c_l x^l \in \mathbb{Z}[x]$, then we obtain $ab = C(2^{64})$. Now $0 \le c_l = \sum_{j+k=l} a_j b_k < \sum_{j+k=l} 2^{128} \le n \cdot 2^{128}$ for all l. We assume that $n < 2^{61}$, take three single precision primes p_1, p_2, p_3 between 2^{63} and 2^{64}, and multiply A and B modulo each p_j. Then the Chinese Remainder Theorem guarantees that we can reconstruct AB from its images modulo the three primes.

For the three modular multiplications, we want to use the FFT multiplication algorithm 8.16, and this requires that each $p_j - 1$ be divisible by a sufficiently high power of 2; we will call such a prime a **Fourier prime**. More precisely, if $t = \lceil \log(2n-1) \rceil$ and 2^t divides $p_j - 1$ for $j = 1, 2, 3$, then $\mathbb{F}_{p_j}$ contains a primitive 2^tth root of unity, by Lemma 8.8, and we may use Algorithm 8.16 with $R = \mathbb{F}_{p_j}$ to compute AB mod p_j. If n is not too large, then three such primes can be found by successively testing $2^t + 1, 2 \cdot 2^t + 1, 3 \cdot 2^t + 1, \ldots$ for primality, using the algorithms from Chapter 18. (Exercise 18.16 shows how to find a primitive 2^tth root of unity modulo such a prime.) For example, for each of the six pairs

k	29	71	75	95	108	123
ω	21	287	149	55	64	493

$p = k \cdot 2^{57} + 1$ is prime and ω is the least positive primitive 2^{57}th root of unity modulo p. In fact, these are all primes p below 2^{64} such that 2^{57} divides $p - 1$, and all but the first one are greater than 2^{63}. (For $p = 108 \cdot 2^{57} + 1$, $p - 1$ is even divisible by 2^{59}; it is the only prime below 2^{64} with that property, and there is no prime p below 2^{64} such that $p - 1$ is divisible by a higher power of 2.) Three such pairs may be precomputed once and for all. Here is the corresponding algorithm.

ALGORITHM 8.25 Three primes FFT integer multiplication.
Input: Two integers $a, b \in \mathbb{N}$ in 2^{64}-adic representation such that $a, b < 2^{64 \cdot 2^{s-1}}$, where $s \le 62$.
Output: $ab \in \mathbb{N}$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

{ we assume three precomputed pairs (p_1,ω_1), (p_2,ω_2), (p_2,ω_3) of single precision integers such that each $p_j \in \{2^{63},\ldots,2^{64}-1\}$ is prime and each $\omega_j \in \{1,\ldots,p_j-1\}$ is a primitive 2^sth root of unity in $\mathbb{F}_{p_j}$ }

1. let $A,B \in \mathbb{Z}[x]$, with all coefficients nonnegative and less than 2^{64}, such that $a = A(2^{64})$ and $b = B(2^{64})$
   $t \longleftarrow \lceil \log_2(1+\deg(AB)) \rceil$
2. **for** $j = 1,2,3$ **call** the fast convolution algorithm 8.16 over $R = \mathbb{F}_{p_j}$ with $\omega \equiv \omega_j^{2^{s-t}} \bmod p_j$ to compute $C_j = AB \text{ rem } p_j$
3. **call** the Chinese Remainder Algorithm 5.4 to compute $C \in \mathbb{Z}[x]$ with nonnegative coefficients less than $p_1p_2p_3$ such that $C \equiv C_j \bmod p_j$ for $j = 1,2,3$
4. **return** $C(2^{64})$

The number s in the algorithm is an implementation dependent constant; it limits the size of the numbers that can be multiplied. By the discussion preceding the algorithm, with primes up to 2^{64} the maximal possible value is $s = 57$, and then the largest integers that can be multiplied by this method have $64 \cdot 2^{56}$ bits or 536870912 gigabytes. The analogous algorithm for 32-bit processors allows factors of 256 megabytes each (Exercise 18.16).

The number $t \leq s$ corresponds to the size of the inputs; the sum of the lengths of a and b in the 2^{64}-ary representation is close to 2^t. The cost for Algorithm 8.25 amounts to essentially nine 2^t-point FFT's of polynomials with single precision coefficients, in total $O(t2^t)$ word operations.

Throughout this text, we discuss algorithms for many problems in computer algebra based on fast polynomial and integer multiplication. In order to abstract from the underlying multiplication algorithm in our cost analyses, we introduce the following notation.

DEFINITION 8.26. *Let R be a ring (commutative, with 1). We call a function $\mathsf{M}\colon \mathbb{N}_{>0} \longrightarrow \mathbb{R}_{>0}$ a **multiplication time for $R[x]$** if polynomials in $R[x]$ of degree less than n can be multiplied using at most $\mathsf{M}(n)$ operations in R. Similarly, a function M as above is called a **multiplication time for** $\mathbb{Z}$ if two integers of length n can be multiplied using at most $\mathsf{M}(n)$ word operations.*

In the last line, $\log^*$ is the number of times that logarithms have to be taken until the result is less than 1. Thus $\log^* n \leq 5$ for $n < 2^{65536}$. For practical purposes, $\log^* n$ behaves like a constant, and Fürer's result comes close to being $O(n \log n)$, which is also conjectured to be optimal.

In principle, any multiplication algorithm leads to a multiplication time. Table 8.6 summarizes the multiplication times for the algorithms we discuss. You find a

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

Algorithm	$\mathsf{M}(n)$
classical	$2n^2$
Karatsuba (Karatsuba & Ofman 1962)	$O(n^{1.59})$
FFT multiplication (provided that R supports the FFT)	$O(n\log n)$
Schönhage & Strassen (1971), Schönhage (1977) Cantor & Kaltofen (1991); FFT based	$O(n\log n\log\log n)$
Fürer (2009); FFT based	$n\log n\cdot 2^{O(\log^* n)}$

TABLE 8.6: Various polynomial multiplication algorithms and their running times.

similar table for quick reference on the inside back cover. In the remainder of this text, we will assume that the multiplication time satisfies

$$\mathsf{M}(n)/n \geq \mathsf{M}(m)/m \text{ if } n \geq m, \quad \mathsf{M}(mn) \leq m^2\mathsf{M}(n), \tag{8}$$

for all $n,m \in \mathbb{N}_{>0}$. The first inequality yields the **superlinearity** properties

$$\mathsf{M}(mn) \geq m\cdot\mathsf{M}(n), \quad \mathsf{M}(n+m) \geq \mathsf{M}(n)+\mathsf{M}(m), \text{ and } \mathsf{M}(n) \geq n \tag{9}$$

for all $m,n \in \mathbb{N}_{>0}$ (Exercise 8.33). The last property in (8) says that M is "at most quadratic" and implies that $\mathsf{M}(cn) \in O(\mathsf{M}(n))$ for all positive constants c. Theorem 8.23 implies that we may take

$$\mathsf{M}(n) \in 63.43\,n\log n\log\log n + O(n\log n)$$

for an arbitrary commutative ring R, and we will mainly use this result.

8.4. Multiplication in $\mathbb{Z}[x]$ and $R[x,y]$

Let $f,g \in \mathbb{Z}[x]$ be of degree less than n. Theorem 8.23 tells us that f and g can be multiplied using $O(n\log n\log\log n)$ additions and multiplications in $\mathbb{Z}$. If we want to determine the number of word operations for their multiplication, we need a bound on the coefficients of the intermediate results. A more detailed analysis of Algorithm 8.20 would provide such an estimate, but we choose a different approach here: **Kronecker substitution**.

Let $f = \sum_{0\leq i<n} f_i x^i$ and $g = \sum_{0\leq j<n} g_j x^j$, with all $f_i, g_j \in \mathbb{Z}$ of length at most l in the 2^{64}-adic representation, and $h = fg = \sum_{0\leq k\leq 2n-2} h_k x^k$, where $h_k = \sum_{i+j=k} f_i g_j$. To simplify our discussion, we assume that all f_i, g_j are nonnegative (this can be achieved, for example, by splitting f and g into their positive and negative parts if necessary). Then

$$0 \leq h_k < \sum_{i+j=k} 2^{128l} \leq n2^{128l}$$

for $0 \leq k \leq 2n-2$. If we choose $t \in \mathbb{N}$ so that $n2^{128l} < 2^{64t}$, then the coefficients of h can be read off the 2^{64}-adic representation of the number $h(2^{64t}) = f(2^{64t})g(2^{64t})$;

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

this corresponds to calculating modulo $x - 2^{64t}$. Thus $t = 2l + \lfloor (\log n)/64 \rfloor + 1 \in O(l + \log n)$ suffices, and then $f(2^{64t})$ and $g(2^{64t})$ have length $O(n(l + \log n))$. Theorem 8.24 now implies the following.

COROLLARY 8.27.
Polynomials in $\mathbb{Z}[x]$ of degree less than n with coefficients of length at most l can be multiplied using $O(\mathsf{M}(n(l + \log n)))$ or $O^{\sim}(nl)$ word operations.

The $O^{\sim}$ notation ignores logarithmic factors; see Section 25.7.

The above result is not easy to implement efficiently. In practice, other modular approaches have proven useful. A method using Fermat numbers is discussed in Exercise 8.36, and we now present a different approach, similar to the three primes FFT algorithm. We choose a collection of distinct single precision primes $p_1, \ldots, p_r \in \mathbb{N}$ such that their product exceeds $n2^{128l+1}$, compute $fg \bmod p_1 \in \mathbb{F}_{p_1}[x], \ldots, fg \bmod p_r \in \mathbb{F}_{p_r}[x]$, and finally recover fg by applying the Chinese Remainder Algorithm 5.4 to each of its coefficients (Exercise 5.34). If furthermore $2n \leq 2^k$ for some $k \in \mathbb{N}$ and we let the p_i be Fourier primes such that $2^k \mid (p_i - 1)$ for all i, then each $\mathbb{F}_{p_i}$ contains a primitive 2^kth root of unity (Lemma 8.8), and $fg \bmod p_i$ can be computed using Algorithm 8.16 directly, without constructing any "virtual" primitive roots of unity. Each p_i fits into one machine word of the target processor, and coefficient arithmetic in $\mathbb{F}_{p^i}$ amounts to just a few machine instructions. The primes p_i and appropriate primitive roots of unity in $\mathbb{F}_{p_i}$ (Exercise 18.16), as well as the "Lagrange interpolants" in the Chinese Remainder Algorithm, which only depend on the p_i, may be precomputed and stored if many polynomial multiplications of the same size have to be performed.

Of course, this approach limits the degree and the coefficient size of the polynomials that can be handled, but it appears to be highly sufficient for practical applications. An implementation by Shoup (1995) and an implementation of the Fermat number approach are described in Section 9.7, and the timings indicate that the former is well suited for small coefficients, while the latter is better when the coefficients get large.

Kronecker substitution also applies to multiplication of bivariate polynomials over a ring R. Let $f = \sum_{0 \leq i < n} f_i x^i, g = \sum_{0 \leq i < n} g_i x^i \in R[y][x]$ be two polynomials, with $f_i, g_i \in R[y]$ of degree less than d for all i. Then $h = fg = \sum_{0 \leq i \leq 2n-2} h_i x^i$, with $h_i \in R[y]$ of degree at most $2d - 2$ for all i. Substituting y^{2d-1} for x (that is, calculating modulo $x - y^{2d-1}$), we obtain

$$f \cdot g \equiv f(y^{2d-1}, y) \cdot g(y^{2d-1}, y) = h(y^{2d-1}, y) = \sum_{0 \leq i \leq 2n-2} h_i y^{(2d-1)i} \equiv h \bmod x - y^{2d-1},$$

and we see that the coefficients of h can be read off the image of fg modulo $x - y^{2d-1}$, which in turn can be computed using fast multiplication of univariate

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

polynomials in $R[y]$. Since the degrees of $f(y^{2d-1}, y)$ and $g(y^{2d-1}, y)$ are at most $(n-1)(2d-1)+d-1 = 2nd-n-d$, we have the following result.

COROLLARY 8.28.
Polynomials in $R[x,y]$, where R is a ring (commutative, with 1), of degree less than n in x and less than d in y can be multiplied using $O(\mathsf{M}(nd))$ or $O^{\sim}(nd)$ ring operations.

Notes. **8.1.** Karatsuba and Ofman participated in a seminar run by Kolmogorov, who had conjectured that the quadratic cost for multiplication cannot be beaten and then actually wrote (probably with Ofman) and submitted their paper for publication. Karatsuba (1995) says about the work that made him famous: *I learned about the article only when I was given its reprints.*

8.2. Cooley & Tukey (1965) discovered the FFT for computer science, causing a revolution in digital signal processing methods. Its original invention goes back a century and a half: Gauß found it around 1805, but it appeared only posthumously (Gauß 1866). Gauß also discovered the usual Fourier Transform, before Fourier did so in 1807 (published in Fourier 1822). The algorithm was rediscovered several times over the years. The fascinating history of the FFT is described in Cooley (1987, 1990) and Heideman, Johnson & Burrus (1984).

8.3. Algorithm 8.20 is an adaption to polynomials of Schönhage & Strassen's (1971) integer multiplication algorithm, where 2 plays the role of our x; Schönhage (1977) solved the additional complication that occurs in characteristic 2 by using a 3-adic FFT (Exercise 8.30). These two papers, and also Cantor & Kaltofen (1991), showed Theorem 8.23. Before Schönhage & Strassen's breakthrough, Toom (1963), Cook (1966), Schönhage (1966), and Strassen (1968, unpublished) had found $n^{1+o(1)}$ multiplication algorithms for integers of length n, but Schönhage & Strassen set the world record that stood for a quarter century, until Fürer (2009). Gentleman & Sande (1966) had earlier proposed to use the FFT for polynomial multiplication. See Knuth (1998), §4.3.3, for a description of some of these methods. An alternative, also briefly discussed there, is to employ the FFT over the complex numbers, with approximations to sufficiently high precision of the complex roots of unity. This is used in calculating π to record accuracy; FFT multiplication uses 90% of the CPU time for such high precision calculations (Kanada (1988); see Section 4.6). Bernstein (2001) gives an exhaustive discussion of fast multiplication routines. Schönhage showed that n-bit integers can be multiplied on random access machines (with cost m to access an m-bit address) using $O(n \log n)$ word operations (see Knuth (1998), §4.3.3 C).

Pollard (1971) presents an FFT-based multiplication algorithm for polynomials over finite fields, over $\mathbb{Z}$, and for integers, including an implementation report, but without asymptotic analysis or a general construction of the required primitive roots of unity. He also gives the three primes FFT algorithm 8.25; see also Lipson (1981), §IX.2.2. Moenck (1976) presents an early implementation report.

For some of the multiplication times that we use, the required properties (8) may fail to hold for $n \leq 2$. We will ignore this systematically.

8.4. Kronecker (1882), §4, invented his substitution for multivariate polynomials; its effect is to keep distinct coefficients "apart" after the substitution. The algorithm for multiplying in $\mathbb{Z}[x]$ modulo several small primes appears in Pollard (1971).

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

Exercises.

8.1 Given the real and imaginary parts $a_0, a_1, b_0, b_1 \in \mathbb{R}$ of two nonzero complex numbers $z_1 = a_0 + a_1 i$ and $z_2 = b_0 + b_1 i$, where $i = \sqrt{-1}$, show how to compute the real and imaginary parts of the quotient $z_1/z_2 \in \mathbb{C}$ using at most 7 multiplications and divisions in $\mathbb{R}$. Draw an arithmetic circuit illustrating your algorithm. Can you achieve at most 6 real multiplications and divisions?

8.2 Let R be a ring (commutative, with 1) and $f, g \in R[x, y]$, and assume that f and g have degrees less than m in y and less than n in x. Let $h = f \cdot g$.

(i) Using classical univariate polynomial multiplication and viewing $R[x, y]$ as $R[y][x]$, bound the number of arithmetic operations in R to compute h.

(ii) Using Karatsuba's algorithm bound the number of operations in R to compute h.

(iii) Generalize parts (i) and (ii) to polynomials in an arbitrary number of variables.

8.3 With notation as in Lemma 8.2, assume that in addition S and T are monotonically increasing and $S(2n) \leq 4S(n)$ for all $n \in \mathbb{N}_{>0}$. Derive a tight upper bound on $T(n)$ that is valid for *all* $n \in \mathbb{N}_{>0}$ and not only for powers of two. Hint: n lies in some interval $[2^{k-1} + 1, 2^k]$.

8.4 We have seen in Section 2.3 that the classical multiplication method for polynomials of degree less than n requires $2n^2 - 2n + 1$ ring operations. For which values of $n = 2^k$ is this larger than the $9 \cdot 3^k - 8 \cdot 2^k$ for Karatsuba's algorithm?

8.5* Let R be a ring (commutative, with 1) and $f, g \in R[x]$ of degrees less than n. You are to analyze two variants of Karatsuba's algorithm 8.1 when n is not a power of two.

(i) The first variant is to call Algorithm 8.1 with $2^{\lceil \log n \rceil}$ instead of n. Show that this takes at most $9 \cdot 3^{\lceil \log n \rceil} \leq 27 n^{\log 3}$ operations in R.

(ii) Let $m = \lceil n/2 \rceil$, and modify Algorithm 8.1 so that it divides f and g into blocks of degree less than m. If $T(n)$ denotes the cost of this algorithm, show that $T(1) = 1$ and $T(n) \leq 3T(\lceil n/2 \rceil) + 4n$.

(iii) Show that $T(2^k) \leq 9 \cdot 3^k - 8 \cdot 2^k$ and $T(2^{k-1} + 1) \leq 6 \cdot 3^k - 4 \cdot 2^k - 2$ for all $k \in \mathbb{N}_{>0}$, and compare this to (i) for $n = 2^k$ and $n = 2^{k-1} + 1$. Plot the curves of the two running time bounds from (i) and (ii) for $n \leq 50$.

(iv) Implement both algorithms for $R = \mathbb{Z}$. Experiment with various values of n, say $2 \leq n \leq 50$ and $n = 100, 200, \ldots, 1000$. Use random polynomials with one-digit coefficients, and also with n-digit coefficients.

8.6* Karatsuba's algorithm is slower for small inputs than classical multiplication. You are to investigate a hybrid algorithm which recursively does Karatsuba until the degrees get smaller than some bound $2^d \in \mathbb{N}$ and then switches to classical multiplication, that is, we replace line 1 of Karatsuba's algorithm 8.1 by

1. **if** $n \leq 2^d$ **then call** Algorithm 2.3 to compute $fg \in R[x]$

Algorithm 8.1 then corresponds to $d = 0$. Let $T(n)$ denote the cost of this algorithm when $n = 2^k$ for some $k \in \mathbb{N}$, and prove that $T(n) \leq \gamma(d) n^{\log 3} - 8n$ holds for $n \geq 2^d$, where $\gamma(d)$ depends only on d. Find the value of $d \in \mathbb{N}$ that minimizes $\gamma(d)$, and compare the result to the running time bound of Theorem 8.3.

8.7 Karatsuba's method for polynomial multiplication can be generalized as follows. Let F be a field, $m, n \in \mathbb{N}_{>0}$, and $f = \sum_{0 \leq i < n} f_i x^i$, $g = \sum_{0 \leq i < n} g_i x^i$ in $F[x]$. To multiply f and g, we divide each of them into $m \geq 2$ blocks of size $k = \lceil n/m \rceil$:

$$f = \sum_{0 \leq i < m} F_i x^{ki}, \quad g = \sum_{0 \leq i < m} G_i x^{ki},$$

with all $F_i, G_i \in F[x]$ of degree less than k. Then $fg = \sum_{0 \leq i < 2m-1} H_i x^{ki}$, where $H_i = \sum_{0 \leq j \leq i} F_j G_{i-j}$ for $0 \leq i < 2m - 1$ and we assume that $F_j, G_j = 0$ if $j \geq m$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

(i) Find a way to compute $H_0, \ldots, H_4$ when $m = 3$ using at most 6 multiplications of polynomials of degree less than k. Use this method to construct a recursive algorithm à la Karatsuba and analyze its cost when n is a power of 3.

(ii) Suppose that you have found a scheme to compute $H_0, \ldots, H_{2m-2}$ using d multiplications of polynomials of degree less than k, and made this scheme into a recursive algorithm as in (i). How large may d be at most such that your algorithm is asymptotically faster than Karatsuba's? Compare with your result from (i).

8.8** This continues Exercise 8.7.

(i) You are to show that $d = 2m-1$ can be achieved provided that the cardinality of F is at least $2m-1$. Let $K = F(\alpha_0, \ldots, \alpha_{m-1}, \beta_0, \ldots, \beta_{m-1})$ be the field of rational functions in $2m$ indeterminates, $\alpha = \sum_{0 \le i < n} \alpha_i x^i$ and $\beta = \sum_{0 \le i < n} \beta_i x^i$ in $K[x]$, $u_0, \ldots, u_{2m-2} \in F$ distinct, and $v_j = \alpha(u_j)\beta(u_j)$ for $0 \le j \le 2m-2$. If

$$l_j = \prod_{\substack{0 \le k \le 2m-2 \\ k \ne j}} \frac{x - u_j}{u_k - u_j} \in F[x]$$

are the Lagrange interpolants at $u_0, \ldots, u_{2m-2}$ for $0 \le j \le 2m-2$, then

$$\alpha\beta = \sum_{0 \le j \le 2m-2} v_j l_j = \sum_{0 \le i \le 2m-2} \gamma_i x^i \in K[x],$$

since $\deg(\alpha\beta) < 2m-1$ and the interpolating polynomial is unique, and each $\gamma_i \in K$ is an F-linear combination

$$\gamma_i = \sum_{0 \le j \le 2m-2} c_{ij} v_j$$

of $v_0, \ldots, v_{2m-2}$, with $c_{ij} \in F$ for $0 \le i, j \le 2m-2$. (In fact, the matrix $(c_{ij})_{0 \le i,j \le 2m-2}$ is the inverse of the Vandermonde matrix $(u_i^j)_{0 \le i,j \le 2m-2}$ in $F^{(2m-1)\times(2m-1)}$.) This yields the following scheme for computing $H_0, \ldots, H_{2m-2}$. We assume that the values u_i^j and c_{ij} for $0 \le i, j \le 2m-2$ are precomputed and stored.

1. Set $P_i = \sum_{0 \le j < m} F_j u_i^j$ and $Q_i = \sum_{0 \le j < m} G_j u_i^j$ for $0 \le i \le 2m-2$.
2. Compute $R_i = P_i Q_i$ for $0 \le i \le 2m-2$.
3. Set $H_i = \sum_{0 \le j \le 2m-2} c_{ij} R_j$ for $0 \le i \le 2m-2$.

Prove that this scheme works correctly (hint: first consider $k = 1$) and figure out the precise number of additions and multiplications in F that steps 1 and 3 take.

(ii) Calculate the values of u_i^j and c_{ij} for $F = \mathbb{F}_5$, $m = 3$, and $u_i = i \bmod 5$ for $0 \le i \le 4$.

(iii) Use the scheme from (i) to construct a recursive algorithm for polynomial multiplication, and determine its asymptotic cost when n is a power of m. Conclude that if F is infinite, then for each positive $\varepsilon \in \mathbb{R}$ there is an algorithm for multiplying polynomials of degree less than n in $F[x]$ taking $O(n^{1+\varepsilon})$ operations in F.

8.9 Let $F = \mathbb{F}_{29}$.

(i) Find a primitive 4th root of unity $\omega \in F$, and compute its inverse $\omega^{-1} \in F$.

(ii) Find the matrices for DFT_ω and $\mathrm{DFT}_{\omega^{-1}}$, and check that their product is $4I$.

8.10⟶ Let $F = \mathbb{F}_{17}$ and $f = 5x^3 + 3x^2 - 4x + 3$, $g = 2x^3 - 5x^2 + 7x - 2$ in $F[x]$.

(i) Show that $\omega = 2$ is a primitive 8th root of unity in F, and compute the inverse $2^{-1} \bmod 17$ of ω in F.

(ii) Compute $h = f \cdot g \in F[x]$.

(iii) For $0 \le j < 8$, compute $\alpha_j = f(\omega^j)$, $\beta_j = g(\omega^j)$, and $\gamma_j = \alpha_j \cdot \beta_j$. Compare γ_j to $h(\omega^j)$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

(iv) Show the two matrices $V_1 = V_\omega$ and $V_2 = 8^{-1}V_{\omega^{-1}}$, and compute their product. Compute the matrix–vector products $V_1 f, V_1 g$, and $V_2 \gamma$, identifying f and g with their coefficient vectors, and with $\gamma = (\gamma_j)_{0 \le j < 8}$ as computed. Comment.

(v) Trace the FFT multiplication algorithm 8.16 to multiply f and g, with ω as above.

8.11$^{\longrightarrow}$ Let $F = \mathbb{F}_{41}$.

(i) Prove that $\omega = 14 \in F$ is a primitive 8th root of unity. Compute all powers of ω, and mark the ones that are primitive 8th roots of unity.

(ii) Let $\eta = \omega^2$, and $f = x^7 + 2x^6 + 3x^4 + 2x + 6 \in F[x]$. Give an explicit calculation of $\alpha = \mathrm{DFT}_\omega(f)$, using the FFT. You only have to do one recursive step, and then can use direct evaluation at powers of η.

(iii) Let $g = x^7 + 12x^5 + 35^3 + 1 \in F[x]$. Compute $\beta = \mathrm{DFT}_\omega(g)$, $\gamma = \alpha \cdot \beta$ with coordinate–wise product, and $h = \mathrm{DFT}_{\omega^{-1}}(\gamma)$.

(iv) Compute $f \cdot g$ in $F[x]$ and $f *_8 g$. Compare with your result from (iii).

8.12$^{\longrightarrow}$ The complex number $\omega = \exp(2\pi i/8) \in \mathbb{C}$, where $i = \sqrt{-1}$, is a primitive 8th root of unity. Let $f = 5x^3 + 3x^2 - 4x + 3$ and $g = 2x^3 - 5x^2 + 7x - 2$ in $\mathbb{C}[x]$, and run the fast convolution algorithm 8.16 on this example to calculate the coefficients of the product $f \cdot g$. (Of course, on such a small example the "fast" algorithm is more tedious than the school method. But, who knows, you may want to multiply polynomials of degree 1 000 000 one day ... .) Multiply linear polynomials by the "classical" method. Use ω only symbolically, with the fact that $\omega^4 = -1$.

8.13 Let R be a ring, $n \in \mathbb{N}_{\ge 1}$, and $\omega \in R$ be a primitive nth root of unity.

(i) Show that ω^{-1} is a primitive nth root of unity.

(ii) If n is even, then show that ω^2 is a primitive $(n/2)$th root of unity. If n is odd, then show that ω^2 is a primitive nth root of unity.

(iii) Let $k \in \mathbb{Z}$ and $d = n/\gcd(n,k)$. Show that ω^k is a primitive dth root of unity; this generalizes both (i) and (ii).

8.14 Let R be a ring, $n \in \mathbb{N}_{\ge 2}$, $\omega \in R$ a primitive nth root of unity, and $\eta \in R$ with $\eta^2 = \omega$. Under what conditions is η a primitive $2n$th root of unity?

8.15* Let $n \in \mathbb{N}_{>0}$ and R be an integral domain of characteristic coprime to n.

(i) Show that the set R_n of all nth roots of unity is a subgroup of the multiplicative group $R^\times$.

(ii) Prove that the following are equivalent for an nth root of unity $\omega \in R$:

(a) ω is a primitive nth root of unity,

(b) $\omega^\ell \ne 1$ for $0 < \ell < n$ (that is, ω has order n in R),

(c) $\omega^\ell \ne 1$ for all $0 < \ell < n$ with $\ell \mid n$,

(d) $\omega^{n/p} \ne 1$ for all prime divisors p of n.

We now assume that R contains a primitive nth root of unity ω.

(iii) Draw all 12th roots of unity for $R = \mathbb{C}$ and mark the primitive ones.

(iv) Show that R_n is cyclic and isomorphic to the additive group $\mathbb{Z}_n$ of integers modulo n (so in particular, $\#R_n = n$). Hint: The polynomial $x^n - 1 \in R[x]$ has at most n roots.

(v) Prove that there are precisely $\varphi(n)$ primitive nth roots of unity, where φ is Euler's totient function.

8.16* Let q be a prime power, $\mathbb{F}_q$ a finite field with q elements, and $n \in \mathbb{N}$ a divisor of $q-1$, with prime factorization $n = p_1^{e_1} \cdots p_r^{e_r}$. For $a \in \mathbb{F}_q^\times$, we denote by $\mathrm{ord}(a)$ the order of a in the multiplicative group $\mathbb{F}_q^\times$, and want to show that $\mathrm{ord}(a) = q-1$ for some $a \in \mathbb{F}_q^\times$. Prove:

(i) $\mathrm{ord}(a) = n$ if and only if $a^n = 1$ and $a^{n/p_j} \ne 1$ for $1 \le j \le r$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

(ii) $\mathbb{F}_q^\times$ contains an element b_j of order $p_j^{e_j}$, for $1 \le j \le r$. Hint: Consider an element of $\mathbb{F}_q^\times$ which is not a root of the polynomial $x^{(q-1)/p_j} - 1$.

(iii) If $a, b \in \mathbb{F}_q^\times$ are elements of coprime orders, then $\mathrm{ord}(ab) = \mathrm{ord}(a)\,\mathrm{ord}(b)$.

(iv) $\mathbb{F}_q^\times$ contains an element of order n.

(v) $\mathbb{F}_q^\times$ is cyclic.

8.17 (i) For all $a \in \mathbb{F}_{19}^\times$, determine the powers a^k with $k \mid 18$, and derive $\mathrm{ord}(a)$ from this data only.

(ii) Determine all $n \in \mathbb{N}_{>0}$ for which $\mathbb{F}_{19}$ contains a primitive nth root of unity, and for each such n, list all primitive nth roots of unity.

8.18 Prove Lemma 8.8. Hint: Exercises 8.15 and 8.16.

8.19* Let $p, q \in \mathbb{N}$ be distinct odd primes, $n = pq$, and $k, l \in \mathbb{N}$.

(i) Given a primitive kth root of unity in $\mathbb{Z}_p^\times$ and a primitive lth root of unity in $\mathbb{Z}_q^\times$, how can you construct a primitive mth root of unity in $\mathbb{Z}_n^\times$, where $m = \mathrm{lcm}(k, l)$?

(ii) Show that $\mathbb{Z}_n^\times$ contains a primitive kth root of unity if and only if $k \mid \mathrm{lcm}(p-1, q-1)$.

(iii) Find primitive 16th roots of unity in $\mathbb{Z}_{17}^\times$ and in $\mathbb{Z}_{97}^\times$, and construct a primitive 16th root of unity in $\mathbb{Z}_{1649}^\times$.

8.20* (i) Let $x, m, n \in \mathbb{N}_{>0}$, $x \ge 2$, $r = n \bmod m$, and $g = \gcd(n, m)$. Prove that $x^n - 1 \bmod x^m - 1 = x^r - 1$, and conclude that $\gcd(x^n - 1, x^m - 1) = x^g - 1$. Answer the same questions when x is an indeterminate.

(ii) Let $n \in \mathbb{N}_{\ge 2}$. The integer $M_n = 2^n - 1$ is the nth **Mersenne number**. Use (i) and Exercise 4.14 to prove that 2 is a primitive nth root of unity in $\mathbb{Z}/\langle M_n \rangle$ if and only if n is prime. Hint: Use Fermat's little theorem 4.9.

8.21 Let F be a field supporting the FFT, and $a, b, q, r \in F[x]$ such that $a = qb + r$ and $\deg r < \deg b \le \deg a < n$ for a power n of 2. We assume that b is coprime to $x^n - 1$. Give an algorithm which on input a, b decides whether $r = 0$, and if so, computes the quotient q using essentially three n-point FFTs.

8.22 Let R be a (commutative) ring supporting the FFT, $f_1, \ldots, f_r, g_1, \ldots, g_r \in R[x]$ polynomials, and $h = \sum_{1 \le j \le r} f_j g_j \in R[x]$, of degree less than a power $n \in \mathbb{N}$ of 2. Prove that h can be computed using $2r + 1$ FFTs of order n plus $O(rn)$ operations in R. Determine the constant hidden in the "O". Compare this result to the time for multiplying each f_j by g_j using the fast convolution algorithm 8.16 and then adding up the products.

8.23$\longrightarrow$ Let $p = 65\,537 = 2^{2^4} + 1$ be the fourth Fermat prime and $\omega = 3$.

(i) Check that $\omega \in \mathbb{F}_p^\times$ is a primitive 2^{16}th root of unity.

(ii) Program classical polynomial multiplication, Karatsuba's algorithm, and the FFT multiplication algorithm 8.16 in $\mathbb{F}_p[x]$ in a computer algebra system of your choice. The inputs and outputs of your program should be the coefficient arrays of two polynomials of degree less than 2^{15} and their product, respectively.

(iii) Design a suitable test series of polynomials with degrees increasing up to 32 767 and determine the crossover points between your implementations of the three algorithms. Compare your timings also to the built-in multiplication routine of the computer algebra system. Create a plot of your timings. Comment on your results.

8.24* We may interleave Algorithms 8.14 and 8.16, as follows. Instead of evaluating f and g at all powers of ω, we reduce the multiplication of f and g modulo $x^n - 1$ to two multiplications modulo $x^{n/2} - 1$. Here is the algorithm:

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

ALGORITHM 8.29 Fast convolution.
Input: $f, g \in R[x]$ of degree less than $n = 2^k$ with $k \in \mathbb{N}$, and the powers $\omega, \omega^2, \ldots, \omega^{n/2} - 1$ of a primitive nth root of unity $\omega \in R$.
Output: $f * g \in R[x]$.

1. **if** $k = 0$ **then return** $f \cdot g \in R$
2. $f_0 \longleftarrow f \text{ rem } x^{n/2} - 1, \quad f_1 \longleftarrow f \text{ rem } x^{n/2} + 1,$
   $g_0 \longleftarrow g \text{ rem } x^{n/2} - 1, \quad g_1 \longleftarrow g \text{ rem } x^{n/2} + 1$
3. **call** the algorithm recursively to compute $h_0, h_1 \in R[x]$ of degrees less than $n/2$ such that
$$h_0(x) \equiv f_0(x) \cdot g_0(x) \bmod x^{n/2} - 1, \quad h_1(\omega x) \equiv f_1(\omega x) \cdot g_1(\omega x) \bmod x^{n/2} - 1$$
4. **return** $\frac{1}{2}((h_0 - h_1)x^{n/2} + h_0 + h_1)$

(i) Prove that the algorithm works correctly and takes $\frac{11}{2} n \log n$ operations in R.

(ii) For small inputs, it is faster to use classical multiplication (or Karatsuba's algorithm) and a subsequent reduction modulo $x^n - 1$ to compute the result. Replace the first line of the above algorithm by

1. **if** $k \leq d$ **then call** Algorithm 2.3 to compute $fg \in R[x]$ and **return** $fg \text{ rem } x^n - 1$

The above algorithm corresponds to $d = 0$. Let $T(n)$ denote the cost of the hybrid algorithm when $n = 2^k$ for some $k \in \mathbb{N}$, and prove that $T(n) = \frac{11}{2} n \log n + \gamma(d) n$ holds for $n \geq 2^d$, where $\gamma(d)$ depends only on d. Find the value of $d \in \mathbb{N}$ which minimizes $\gamma(d)$, and compare the result to (i).

8.25* Algorithm 8.14 computes DFT_ω over a (commutative) ring R by dividing the input polynomial $f \in R[x]$ of degree less than n by $x^{n/2} - 1$ and $x^{n/2} + 1$ with remainder. A different approach is to split f into its odd and even parts, that is, to write $f = f_0(x^2) + x f_1(x^2)$ with $f_0, f_1 \in R[x]$ of degree less than $n/2$, and then to compute $\text{DFT}_{\omega^2}(f_0)$ and $\text{DFT}_{\omega^2}(f_1)$ recursively. Work out the details and prove that your algorithm uses $cn \log n$ operations in R for some positive constant $c \in \mathbb{Q}$ when n is a power of 2. Modify, if necessary, your algorithm so that $c = 3/2$, as in Theorem 8.15. Hint: Use $\omega^{n/2} = -1$. Draw an arithmetic circuit illustrating your algorithm for $n = 8$, and compare it to the circuit in Figure 8.4.

8.26* Let R be a ring (commutative, with 1) containing a primitive 3^kth root of unity for any $k \in \mathbb{N}$.

(i) Design a 3-adic FFT algorithm, taking as input $k \in \mathbb{N}$, a polynomial $f \in R[x]$ of degree less than $n = 3^k$, and a list of powers $1, \omega, \omega^2, \ldots, \omega^{n-1}$ of a primitive nth root of unity $\omega \in R$, and returning $f(1), f(\omega), f(\omega^2), \ldots, f(\omega^{n-1})$. Prove the correctness of your algorithm. Hint: Consider dividing f with remainder by $x^{n/3} - 1$, $x^{n/3} - \omega^{n/3}$, and $x^{n/3} - \omega^{2n/3}$.

(ii) Draw an arithmetic circuit illustrating your algorithm for $n = 9$.

(iii) Let $T(n)$ denote the cost of your algorithm in operations in R when $n = 3^k$ for some $k \in \mathbb{N}$. Set up a recursion for $T(n)$ (don't forget the initial condition) and solve it.

(iv) Assuming that R contains primitive nth root of unity for any $n \in \mathbb{N}$, generalize the above to an m-adic FFT algorithm for arbitrary $m \in \mathbb{N}_{\geq 2}$.

(v) Formulate an alternative m-adic FFT algorithm as in Exercise 8.25.

8.27 Let F be a field containing a primitive 2^kth root of unity for all $k \in \mathbb{N}$. Algorithm 8.16 shows that the convolution $*_n$ in $F[x]$ can be computed with $O(n \log n)$ operations in F if n is a power of 2. The goal of this exercise is to generalize this to arbitrary $n \in \mathbb{N}$. So let $f, g \in F[x]$ and $m \in \mathbb{N}$ be a power of 2 such that $m/2 < 2n \leq m$, and set $a = f \cdot (x^{m-n} + 1)$ and $b = g$. Show how to obtain the coefficients of $f *_n g$ from those of $a *_m b$, and derive the claim from this.

8.28 Show that $\omega = x \bmod (x^n - 1) \in R = F[x]/\langle x^n - 1 \rangle$, where F is a field of characteristic not dividing n, is *not* a primitive nth root of unity for $n \geq 2$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

8.29* Let R be a ring (commutative, with 1).

(i) For $p \in \mathbb{N}_{\geq 2}$, determine the quotient and remainder on division of $f_p = x^{p-1} + x^{p-2} + \cdots + x + 1$ by $x - 1$ in $R[x]$. Conclude that $x - 1$ is invertible modulo f_p if p is a unit in R and that $x - 1$ is a zero divisor modulo f_p if p is a zero divisor in R.

(ii) Assume that 3 is a unit in R, and let $n = 3^k$ for some $k \in \mathbb{N}$, $D = R[x]/\langle x^{2n} + x^n + 1\rangle$, and $\omega = x \bmod x^{2n} + x^n + 1 \in D$. Prove that $\omega^{3n} = 1$ and $\omega^n - 1$ is a unit. Hint: Calculate $(\omega^n + 2)(\omega^n - 1)$. Conclude that ω is a primitive $3n$th root of unity.

(iii) Let $p \in \mathbb{N}$ be prime and a unit in R, $n = p^k$ for some $k \in \mathbb{N}$, $\Phi_{pn} = f_p(x^n) = x^{(p-1)n} + x^{(p-2)n} + \cdots + x^n + 1 \in R[x]$ the pnth cyclotomic polynomial, $D = R[x]/\langle \Phi_{pn}\rangle$, and $\omega = x \bmod \Phi_{pn} \in D$. Prove that $\omega^{pn} = 1$ and $\omega^n - 1$ is a unit. Hint: Calculate $(\omega^{(p-2)n} + 2\omega^{(p-3)n} + \cdots + (p-2)\omega^n + (p-1)) \cdot (\omega^n - 1)$. Conclude that ω is a primitive pnth root of unity.

8.30** In this exercise, we discuss Schönhage's (1977) 3-adic variant of Algorithm 8.20. It works over any (commutative) ring R such that 3 is a unit in R, in particular over a field of characteristic 2.

ALGORITHM 8.30 Schönhage's algorithm.

Input: Two polynomials $f, g \in R[x]$ of degree less than $2n = 2 \cdot 3^k$ for some $k \in \mathbb{N}$, where R is a (commutative) ring and 3 is a unit in R.

Output: $h \in R[x]$ such that $fg \equiv h \bmod (x^{2n} + x^n + 1)$ and $\deg h < 2n$.

1. **if** $k \leq 2$ **then**
   call the classical algorithm 2.3 (or Karatsuba's algorithm 8.1) to compute $f \cdot g$
   return $fg \operatorname{rem} x^{2n} + x^n + 1$
2. $m \longleftarrow 3^{\lceil k/2 \rceil}, \quad t \longleftarrow n/m$
   let $f', g' \in R[x,y]$ with $\deg_x f', \deg_x g' < m$ such that $f = f'(x, x^m)$ and $g = g'(x, x^m)$
3. let $D = R[x]/\langle x^{2m} + x^m + 1\rangle$
   if $m = t$ **then** $\eta \longleftarrow x \bmod (x^{2m} + x^m + 1)$ **else** $\eta \longleftarrow x^3 \bmod (x^{2m} + x^m + 1)$
   { η is a primitive $3t$th root of unity }
   $f^* \longleftarrow f' \bmod (x^{2m} + x^m + 1), \quad g^* \longleftarrow g' \bmod (x^{2m} + x^m + 1)$
4. **for** $j = 1, 2$ **do**
   $f_j \longleftarrow f^* \operatorname{rem} y^t - \eta^{jt}, \quad g_j \longleftarrow g^* \operatorname{rem} y^t - \eta^{jt}$
   call the fast convolution algorithm 8.16 with $\omega = \eta^3$ to compute $h_j \in D[y]$ of degrees less than t such that
   $$f_j(\eta^j y) g_j(\eta^j y) \equiv h_j(\eta^j y) \bmod y^t - 1$$
   { the DFTs are performed by the 3-adic FFT algorithm from Exercise 8.26, and Algorithm 8.30 is used recursively for multiplications of elements in D }
5. $h^* \longleftarrow \frac{1}{3}(y^t(h_2 - h_1) + \eta^{2t} h_1 - \eta^t h_2)(2\eta^t + 1)$
   let $h' \in R[x,y]$ with $\deg_x h' < 2m$ such that $h^* = h' \bmod (x^{2m} + x^m + 1)$
   $h \longleftarrow h'(x, x^m) \operatorname{rem} (x^{2n} + x^n + 1)$
   return h

(i) Use Exercise 8.29 to prove that the algorithm works correctly.

(ii) Let $T(k)$ denote the cost of the algorithm for $n = 3^k$. Prove that $T(k) \leq 2 \cdot 3^{\lfloor k/2 \rfloor} T(\lceil k/2 \rceil) + (c + 48(\lfloor k/2 \rfloor + 1/2))3^k$ for $k > 2$ and some constant $c \in \mathbb{N}$, and conclude that $T(k)$ is at most $24 \cdot 3^k \cdot k \cdot \log k + O(3^k \cdot k) = 24 n \log_3 n \log_2 \log_3 n + O(n \log n)$. Hint: Consider the function $S(k) = (3^{-k} T(k) + c)/(k-1)$, and prove that $S(k) \leq S(\lceil k/2 \rceil) + 24$ if $k > 2$.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

8.31* (i) Let R be a (commutative) ring, $n \in \mathbb{N}_{\geq 2}$ a power of two, and $\omega = (x \bmod x^{n/2}+1) \in R[x]/\langle x^{n/2}+1\rangle$. Show that the conclusion $V_\omega \cdot V_{\omega^{-1}} = nI$ of Theorem 8.13 holds even when n is not a unit in R. Hint: Show first that $\omega^{nt/2}+1=0$ for all odd $t \in \mathbb{N}$ and use the factorization

$$\sum_{0\leq j<n} x^j = (x+1)(x^2+1)(x^4+1)\cdots(x^{n/2}+1)$$

in $R[x]$. Generalize this to the case when n is an arbitrary prime power.

(ii) Let $e(k)$ be such that the division-free variant of Algorithm 8.20 described in the text returns $2^{e(k)}h$ on input $n=2^k$ for $k \geq 2$. Prove that $e(k+1) = e(\lceil k/2\rceil+1)+\lfloor k/2\rfloor+1$ for $k \geq 2$, and conclude that $e(k) = k-2+\lceil \log(k-1)\rceil$.

8.32* Prove that for $n \in \mathbb{N}_{\geq 1}$, $\omega = 2$ is a primitive $2n$th root of unity modulo 2^n+1 if and only if n is a power of 2.

8.33 Prove (9).

8.34 Let M be a multiplication time for polynomials. Prove that $\mathsf{M}(n+1) \leq \mathsf{M}(n)+4n$ for all n.

8.35 Let $k,n \in \mathbb{N}$.

(i) Prove that a polynomial of degree less than n can be multiplied by a polynomial of degree less than kn in time $k\mathsf{M}(n)+O(kn)$. Determine a "small" value for the constant in the "O".

(ii) Now let n and k be powers of 2. For which values of k is your method from (i) faster than the naive $\mathsf{M}(kn)$? Answer the latter question for the classical $\mathsf{M}(n)=2n^2$, Karatsuba's $\mathsf{M}(n)=9n^{\log 3}$, and $\mathsf{M}(n)=\frac{9}{2}n\log n$ from Theorem 8.18.

8.36* Let $k,l \in \mathbb{N}$ and $f,g \in \mathbb{Z}[x]$ with $\deg(fg) < n = 2^k$, $\|f\|_\infty, \|g\|_\infty \leq 2^l$, and $2l+1 \leq n-k$.

(i) Show that $\|fg\|_\infty < 2^{n-1}$.

(ii) Use Exercise 8.32 and the fast convolution algorithm 8.16 over $R = \mathbb{Z}/\langle 2^n+1\rangle$ to compute $fg \in \mathbb{Z}[x]$, and show that this takes $O(n^2 \log n \operatorname{loglog} n)$ word operations. (This is the essential part of Schönhage & Strassen's (1971) fast integer multiplication algorithm; they call the algorithm recursively for the coefficient multiplications.)

8.37* Adapt Exercise 8.36 to bivariate polynomials.

8.38 Generalize the Kronecker substitution to reduce multiplication of r-variable polynomials to multiplication of univariate polynomials, and analyze its cost, including the dependency on r.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press

What we know is a drop, what we don't know, an ocean.

Isaac Newton

Mit Ausnahme der paar von Hand gefertigten Möbel, Kleider, Schuhe und der Kinder erhalten wir alles unter Einschaltung mathematischer Berechnungen. Dieses ganze Dasein, das um uns läuft, rennt, steht, ist nicht nur für seine Einsehbarkeit von der Mathematik abhängig, sondern ist effektiv durch sie entstanden.[1]

Robert Musil (1913)

Hat man diesen Gegenstand [*die imaginären Grössen*] bisher aus einem falschen Gesichtspunkt betrachtet und eine geheimnissvolle Dunkelheit dabei gefunden, so ist diess grossentheils den wenig schicklichen Benennungen zuzuschreiben. Hätte man $+1, -1, \sqrt{-1}$ nicht positive, negative, imaginäre (oder gar unmögliche) Einheit, sondern etwa directe, inverse, laterale Einheit genannt, so hätte von einer solchen Dunkelheit kaum die Rede sein können.[2]

Carl Friedrich Gauß (1831)

Before the introduction of the Arabic notation, multiplication was difficult, and the division even of integers called into play the highest mathematical faculties. Probably nothing in the modern world could have more astonished a Greek mathematician than to learn that [...] a large proportion of the population of Western Europe could perform the operation of division for the largest numbers.

Alfred North Whitehead (1911)

Dixit Alchoarizmi: [...] Hec sunt igitur universa, que necessaria sunt hominibus ex divisione et multiplicatione in integro numero et in ceteris, que secuntur. His peractis incipiemus narrare multiplicationem fractionum et divisionem earum sive radices, si deus voluerit.[3]

Abū Jaʿfar Muḥammad bin Mūsā al-Khwārizmī (c. 830)

[1] Except for a few hand-made pieces of furniture, clothes, shoes, and our children we obtain everything by using mathematical computations. The whole world we live in, everything which walks, runs, stands around us, depends not only for our understanding on mathematics, but has effectively been created by it.

[2] That this subject [*the imaginary magnitudes*] has hitherto been considered from the wrong point of view and surrounded by a mysterious obscurity, is to be attributed largely to an unfortunate notation. If $+1$, -1, $\sqrt{-1}$ had not been called positive, negative, and imaginary (or even impossible) unit, but rather direct, inverse, and lateral unit, then such an obscurity would probably not have arisen.

[3] Thus spake Al-Khwārizmī: [...] So this is everything that is necessary for men concerning the division and multiplication with an integer, and the other things that are connected with it. Having completed this, we now begin to discuss the multiplication of fractions and their division, and the extraction of roots, if God so wills.

https://doi.org/10.1017/CBO9781139856065.011 Published online by Cambridge University Press